PT3S Documentation

Release 90.11.01.07.dev1

3S

Mar 15, 2019

CONTENTS

Use SIR 3S Modeldata and SIR 3S Calculationresults in pure Python. With pandas, matplotlib and others. For documentation, test, verification, analysis, reporting, prototyping, play.

CHAPTER

ONE

README

The mandatory README.rst File.

Version-String in conf.py and setup.py

sphinx-build -b html . _build was used to build html-doc

sphinx-build -b latex . _build was used to build latex

pdflatex –extra-mem-bot=10000000 PT3S.tex was used to build PDF (executed in _build; PT3S.PDF is generated in _build; copied to ..)

CHAPTER

TWO

MX

SIR 3S MX-Interface (short: MX)

MX is a file based, channel-oriented interface for SIR 3S' calculation results.

This module contains stuff to utilize SIR 3S' MX calculation results in pure Python.

SIR 3S MX calculation results overview:

- Binary .MXS-Files contain the SIR 3S calculations results.
- A Model calculation run creates at least one .MXS-File.
- There is one .MX1-File (an XML-File) for the corresponding .MXS-File(s).
- This .MX1-File defines (in XML) a sequence of MX-Channels in the corresponding .MXS-File(s).
- And - as a result - the Byte-Layout of a single Record in the corresponding .MXS-File(s).
- A single Record is called a MX3-Record. .MXS-File(s) contain a sequence of MX3-Records.
- A MX3-Record contains calculation results for one TIMESTAMP. TIMESTAMP ist Scenariotime.
- A corresponding .MX3-File contains one MX3-Record - the last Scenariotime calculated.
- Summary:
- .MXS-File(s): MX3-Records: MX-Channels
- MX3-Record Byte-Layout (the MX-Channels) defined in corresponding .MX1-File.

A MX-Channel can be:

- a single Value
- or a Vector: Sequence of Values of the same Type:
  - for all Objects of a certain Type or (called Vectorchannels)
  - number of interior Points for all Pipes (special Vectorchannels: Pipevectorchannels)
  - Vectors with ATTRTYPE in: {'SVEC', 'PVECMIN_INST', 'PVECMAX_INST'}

For Vectorchannels (including Pipevectorchannels) the sequence of Objects is defined in the .MX2-File.

```
>>> # ---
>>> # SETUP
>>> # ---
>>> import os
>>> import time
>>> import logging
>>> logger = logging.getLogger('PT3S.Mx')
```

(continues on next page)

(continued from previous page)

```
>>> # ---
>>> # path
>>> # ---
>>> if __name__ == "__main__":
...     try:
...         dummy=__file__
...         logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ','path =␣
↪os.path.dirname(__file__)'," ."))
...         path = os.path.dirname(__file__)
...     except NameError:
...         logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ","path =
↪'.' because __file__ not defined and: "," from Mx import Mx"))
...         path = '.'
...         from Mx import Mx
... else:
...     logger.debug("{0:s}{1:s}{2:s}{3:s}".format('DOCTEST: Not __main__ Context: ','_
↪_name__: ',__name__,"path = '."))
...     path = '.'
>>> # ---
>>> # testDir
>>> # ---
>>> # globs={'testDir':'testdata'}
>>> try:
...      dummy= testDir
... except NameError:
...      testDir='testdata'
>>> # ---
>>> # dotResolution
>>> # ---
>>> # globs={'dotResolution':''}
>>> try:
...      dummy= dotResolution
... except NameError:
...      dotResolution=''
>>> import zipfile
>>> import pandas as pd
>>> # ---
>>> # Init
>>> # ---
>>> h5File=os.path.join(path,os.path.join(testDir,'OneLPipe.h5'))
>>> mx1File=os.path.join(path,os.path.join(testDir,'WDOneLPipe\B1\V0\BZ1\M-1-0-1
↪'+dotResolution+'.MX1'))
>>> mx=Mx(mx1File=mx1File,NoH5Read=True,NoMxsRead=True)
>>> isinstance(mx.mx1Df,pd.core.frame.DataFrame) # MX1-Content
True
>>> isinstance(mx.df,type(None)) # MXS-Content
True
>>> # ---
>>> # Clean Up
>>> # ---
>>> if os.path.exists(mx.h5File):
...     os.remove(mx.h5File)
>>> metadataFile=mx.h5File+'.metadata'
>>> if os.path.exists(metadataFile):
...     os.remove(metadataFile)
>>> if os.path.exists(mx.mxsZipFile):
...     os.remove(mx.mxsZipFile)
```

(continues on next page)

(continued from previous page)

```
>>> mxsDumpFile=mx.mxsFile+'.dump'
>>> if os.path.exists(mxsDumpFile):
...     os.remove(mxsDumpFile)
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> # ---
>>> # 1st Read MXS
>>> # ---
>>> logger.debug("{0:s}: 1st Read MXS".format('DOCTEST'))
>>> mx.setResultsToMxsFile() # looks for M-1-0-1.MXS in same Dir
4
>>> isinstance(mx.df,pd.core.frame.DataFrame) # MXS-Content
True
>>> rowsDf,colsDf = mx.df.shape
>>> (firstTime,lastTime,rows)=mx._checkMxsVecsFile()
>>> rowsDf==rows
True
>>> mx.df.index[0]==firstTime
True
>>> # ---
>>> # Write H5
>>> # ---
>>> if os.path.exists(mx.h5File):
...       os.remove(mx.h5File)
>>> mx.ToH5() # M-1-0-1.h5 in same Dir
>>> os.path.exists(mx.h5File)
True
>>> # ---
>>> # Init with H5
>>> # ---
>>> mx=Mx(mx1File=mx1File) # looks for M-1-0-1.h5 in same Dir
>>> # and reads the .h5 if newer than .MX1 and newer than an existing .MXS
>>> isinstance(mx.mx1Df,pd.core.frame.DataFrame) # MX1-Content
True
>>> isinstance(mx.df,pd.core.frame.DataFrame) # MXS-Content
True
>>> # ---
>>> # 1st Read MXS Zip
>>> # ---
>>> # create the Zip first
>>> with zipfile.ZipFile(mx.mxsZipFile,'w') as myzip:
...     myzip.write(mx.mxsFile)
>>> logger.debug("{0:s}: 1st Read MXS Zip".format('DOCTEST'))
>>> mx.setResultsToMxsZipFile() # looks for M-1-0-1.ZIP in same Dir
4
>>> isinstance(mx.df,pd.core.frame.DataFrame) # MXS-Content
True
>>> rowsMxs,colsMxs = mx.df.shape
>>> mx.df.index.is_unique # all setResultsTo... will ensure this uniqueness under all␣
↪circumstances
True
>>> # uniqueness under all circumstances: also when add=True (setResultsTo... shall␣
↪add the MXS-Content) is used
>>> # ---
>>> # 1st Add same MXS (for testing ensuring uniqueness)
>>> # ---
>>> oldShape=mx.df.shape
```

(continues on next page)

(continued from previous page)

```
>>> logger.debug("{0:s}: 1st Add same MXS (for testing ensuring uniqueness)".format(
↪'DOCTEST'))
>>> mx.setResultsToMxsFile(add=True) # looks for M-1-0-1.MXS in same Dir
0
>>> newShape=mx.df.shape
>>> newShape==oldShape
True
>>> # ---
>>> # 1st Add same Zip (for testing ensuring uniqueness)
>>> # ---
>>> logger.debug("{0:s}: 1st Add same Zip (for testing ensuring uniqueness)".format(
↪'DOCTEST'))
>>> mx.setResultsToMxsZipFile(add=True) # looks for M-1-0-1.ZIP in same Dir
0
>>> newShape=mx.df.shape
>>> newShape==oldShape
True
>>> # ---
>>> # 1st Read MXS Zip with overlapping Timestamps (for testing ensuring uniqueness)
>>> # ---
>>> with zipfile.ZipFile(mx.mxsZipFile,'w') as myzip:
...     myzip.write(mx.mxsFile)
...     myzip.write(mx.mxsFile,arcname=mx.mxsFile+'.2')
>>> logger.debug("{0:s}: 1st Read MXS Zip with overlapping Timestamps (for testing␣
↪ensuring uniqueness)".format('DOCTEST'))
>>> mx.setResultsToMxsZipFile() # looks for M-1-0-1.ZIP in same Dir
4
>>> newShape=mx.df.shape
>>> newShape==oldShape
True
>>> # ---
>>> # shift to younger Timestamps (for testing purposes)
>>> # ---
>>> lastTimestamp=mx.df.index[-1]
>>> firstTimestamp=mx.df.index[0]
>>> timeSpan=lastTimestamp-firstTimestamp
>>> if len(mx.df.index)>1:
...     timeStep=mx.df.index[-1]-mx.df.index[-2]
... else:
...     timeStep=pd.to_timedelta('1 second')
>>> mx.df.index=mx.df.index-(timeSpan+timeStep)
>>> # ---
>>> # 1st Read MXS (with the original Timestamps)
>>> # ---
>>> logger.debug("{0:s}: 1st Read MXS (with the original Timestamps)".format('DOCTEST
↪'))
>>> mx.setResultsToMxsFile(add=True) # looks for M-1-0-1.MXS in same Dir
4
>>> rowsNew,colsNew=mx.df.shape
>>> rowsOld,colsOld=oldShape
>>> rowsNew==2*rowsOld
True
>>> colsNew==colsOld
True
>>> # ---
>>> # shift to older Timestamps (for testing purposes)
>>> # ---
```

(continues on next page)

(continued from previous page)

```
>>> lastTimestamp=mx.df.index[-1]
>>> firstTimestamp=mx.df.index[0]
>>> timeSpan=lastTimestamp-firstTimestamp
>>> if len(mx.df.index)>1:
...     timeStep=mx.df.index[-1]-mx.df.index[-2]
... else:
...     timeStep=pd.to_timedelta('1 second')
>>> mx.df.index=mx.df.index+(timeSpan+timeStep)
>>> # ---
>>> # 2nd Read MXS (with the original Timestamps)
>>> # ---
>>> logger.debug("{0:s}: 2nd Read MXS (with the original Timestamps)".format('DOCTEST
↪'))
>>> mx.setResultsToMxsFile(add=True) # looks for M-1-0-1.MXS in same Dir
4
>>> rowsNew,colsNew=mx.df.shape
>>> rowsNew==3*rowsOld
True
>>> colsNew==colsOld
True
>>> # ---
>>> # Write Dump
>>> # ---
>>> mx.dumpInMxsFormat() # dumps to .MXS.dump-File in same Dir
(12, 8)
>>> # ---
>>> # Read Dump
>>> # ---
>>> logger.debug("{0:s}: Read Dump".format('DOCTEST'))
>>> mx.setResultsToMxsFile(mxsFile=mxsDumpFile)
12
>>> with zipfile.ZipFile(mx.mxsZipFile,'w') as myzip:
...     myzip.write(mx.mxsFile)
...     myzip.write(mxsDumpFile)
>>> # ---
>>> # Read Zip with Orig and Dump
>>> # ---
>>> logger.debug("{0:s}: Read Zip with Orig and Dump".format('DOCTEST'))
>>> mx.setResultsToMxsZipFile()
12
>>> (rows,cols)=mx.df.shape
>>> rows
12
>>> mx.ToH5()
>>> # ---
>>> # Without MX1, MXS
>>> # ---
>>> os.rename(mx.mx1File,mx.mx1File+'.blind')
>>> os.rename(mx.mxsFile,mx.mxsFile+'.blind')
>>> mx=Mx(mx1File=mx1File)
>>> os.rename(mx.mx1File+'.blind',mx.mx1File)
>>> os.rename(mx.mxsFile+'.blind',mx.mxsFile)
>>> # ---
>>> sir3sIdTimestamp=mx.mx1Df['Sir3sID'].iloc[mx.idxTIMESTAMP]
>>> mx.mx1Df['Sir3sID'][mx.mx1Df['Sir3sID']==sir3sIdTimestamp].index[0]
0
>>> mx.mx1Df['unpackIdx'][mx.mx1Df['Sir3sID']==sir3sIdTimestamp].iloc[0]
```

(continues on next page)

(continued from previous page)

```
0
>>> (rows,cols)=mx.df.shape
>>> rows
12
>>> isinstance(mx.df.index[0],pd.tslib.Timestamp)
True
>>> str(mx.df.index[0])
'2018-03-03 00:00:00+00:00'
>>> sir3sId=mx.mx1Df['Sir3sID'][mx.mx1Df['Sir3sID'].str.contains('KNOT~\S*~~
↪5642914844465475844~QM')].iloc[0] #KNOT~I~~5642914844465475844~QM
>>> ts=mx.df[sir3sId]
>>> isinstance(ts,pd.core.series.Series)
True
>>> "{:06.2f}".format(round(ts.iloc[0],2))
'176.71'
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.31',1,'New','_checkMxsVecsFile: (...,fullCheck=False,...)'))
>>> mx._checkMxsVecsFile()
(Timestamp('2018-03-03 00:00:00+0000', tz='UTC'), Timestamp('2018-03-03 00:00:11+0000
↪', tz='UTC'), 12)
>>> mx._checkMxsVecsFile(fullCheck=True)
(Timestamp('2018-03-03 00:00:00+0000', tz='UTC'), Timestamp('2018-03-03 00:00:11+0000
↪', tz='UTC'), 12)
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',1,'Change','setResultsToMxsFile: finally: h5.close()'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',2,'Change','setResultsToMxsZipFile: finally: h5.close()'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',3,'Change','ToH5: finally: h5.close()'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',4,'Change','FromH5: finally: h5.close()'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',5,'Change','*: except Exception as e'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',6,'Change','setResultsToMxsFile: finally: NewH5Vec=False'))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',7,'Change','__init__(...,NoH5Read=True,...)'))
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> mx=Mx(mx1File=mx1File,NoH5Read=True)
>>> os.path.exists(mx.h5FileVecs) # h5 was written (h5 did not exist)
True
>>> h5VecsFileTime=os.path.getmtime(mx.h5FileVecs)
>>> mx=Mx(mx1File=mx1File,NoH5Read=True) # h5 will be written again (h5 exists)
>>> h5VecsFileTimeNow=os.path.getmtime(mx.h5FileVecs)
>>> logger.debug("1 h5VecsFileTime:{:s} < h5VecsFileTimeNow:{:s}".format(time.
↪strftime("%Y-%m-%d %H:%M:%S %z",time.gmtime(h5VecsFileTime)),time.strftime("%Y-%m-
↪%d %H:%M:%S %z",time.gmtime(h5VecsFileTimeNow))))
>>> h5VecsFileTime<h5VecsFileTimeNow # 1 h5 was written again (h5 did exist before)
True
>>> h5VecsFileTime=os.path.getmtime(mx.h5FileVecs)
>>> mx=Mx(mx1File=mx1File) # h5 is read - not written
>>> h5VecsFileTimeNow=os.path.getmtime(mx.h5FileVecs)
>>> logger.debug("2 h5VecsFileTime:{:s} == h5VecsFileTimeNow:{:s}".format(time.
↪strftime("%Y-%m-%d %H:%M:%S %z",time.gmtime(h5VecsFileTime)),time.strftime("%Y-%m-
↪%d %H:%M:%S %z",time.gmtime(h5VecsFileTimeNow))))
>>> mx.setResultsToMxsFile() # h5 will not be updated
```

(continues on next page)

(continued from previous page)

```
0
>>> h5VecsFileTimeNow=os.path.getmtime(mx.h5FileVecs)
>>> logger.debug("3 h5VecsFileTime:{:s} == h5VecsFileTimeNow:{:s}".format(time.
↪strftime("%Y-%m-%d %H:%M:%S %z",time.gmtime(h5VecsFileTime)),time.strftime("%Y-%m-
↪%d %H:%M:%S %z",time.gmtime(h5VecsFileTimeNow))))
>>> mx.setResultsToMxsFile(NewH5Vec=True) # h5 is written
4
>>> h5VecsFileTimeNow=os.path.getmtime(mx.h5FileVecs)
>>> logger.debug("h5VecsFileTime:{:s} == h5VecsFileTimeNow:{:s}".format(time.strftime(
↪"%Y-%m-%d %H:%M:%S %z",time.gmtime(h5VecsFileTime)),time.strftime("%Y-%m-%d %H:%M:
↪%S %z",time.gmtime(h5VecsFileTimeNow))))
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.32',8,'Change','setResultsToMxsZipFile: finally: NewH5Vec=False'))
>>> mx.ToH5()
>>> mx=Mx(mx1File=mx1File,NoH5Read=True)
>>> os.path.exists(mx.h5File)
False
>>> pd.set_option('display.max_columns',None)
>>> pd.set_option('display.max_rows',None)
>>> pd.set_option('display.max_colwidth',666666)
>>> pd.set_option('display.width',666666666)
>>> print(mx._getMx1DfAsOneString().replace('\n','\n '))
ATTRTYPE  DATALENGTH DATATYPE  DATATYPELENGTH  FLAGS OBJTYPE           OBJTYPE_PK    ␣
↪           TITLE    UNIT
      PH           4     REAL               4   1265    KNOT  5289899964753656852    ␣
↪           Druck   [bar]
      QM           4     REAL               4   1265    KNOT  5289899964753656852 ␣
↪Externer Durchfluss  [m3/h]
      PH           4     REAL               4   1265    KNOT  5642914844465475844    ␣
↪           Druck   [bar]
      QM           4     REAL               4   1265    KNOT  5642914844465475844 ␣
↪Externer Durchfluss  [m3/h]
>>> print("'''{:s}'''".format(repr(mx.mx2Df).replace('\n','\n   ')))
'''       AttrType                                        Data  DataLength DataType ␣
↪DataTypeLength  NOfItems       ObjType
   0  tk            [5642914844465475844, 5289899964753656852]          40     CHAR ␣
↪           20         2  KNOT
   1  pk                                  [5252810657060947333]          20     CHAR ␣
↪           20         1  LFKT
   2  pk                                  [5502689500012692689]          20     CHAR ␣
↪           20         1  PHI1
   3  pk                                  [5732781659713982525]          20     CHAR ␣
↪           20         1  PUMD
   4  pk                                  [5163733225086798083]          20     CHAR ␣
↪           20         1  PVAR
   5  pk                                  [4742976321174242828]          20     CHAR ␣
↪           20         1  QVAR
   6  tk                                  [4737064599036143765]          20     CHAR ␣
↪           20         1  ROHR
   7  N_OF_POINTS                                        (101,)           4     INT4 ␣
↪            4         1  ROHR
   8  pk                                  [5396761270498593493]          20     CHAR ␣
↪           20         1  SWVT        '''
>>> print(mx._getDfAsOneString())
                         KNOT~~~5289899964753656852~PH KNOT~~~5289899964753656852~QM␣
↪KNOT~~~5642914844465475844~PH KNOT~~~5642914844465475844~QM
2018-03-03 00:00:00+00:00                          0.0                          -176.7␣
↪                          4.2                            176.7
```

(continues on next page)

(continued from previous page)

```
2018-03-03 00:00:01+00:00                                0.0                          -176.7␣
↪                          4.2                              176.7
2018-03-03 00:00:02+00:00                                0.0                          -176.7␣
↪                          4.2                              176.7
2018-03-03 00:00:03+00:00                                0.0                          -176.7␣
↪                          4.2                              176.7
>>> # ---
>>> # Clean Up OneLPipe
>>> # ---
>>> if os.path.exists(mx.h5File):
...     os.remove(mx.h5File)
>>> metadataFile=mx.h5File+'.metadata'
>>> if os.path.exists(metadataFile):
...     os.remove(metadataFile)
>>> if os.path.exists(mx.mxsZipFile):
...     os.remove(mx.mxsZipFile)
>>> if os.path.exists(mxsDumpFile):
...    os.remove(mxsDumpFile)
>>> if os.path.exists(mx.h5FileVecs):
...    os.remove(mx.h5FileVecs)
>>> # ---
>>> # LocalHeatingNetwork
>>> # ---
>>> mx1File=os.path.join(path,os.path.join(testDir,'WDLocalHeatingNetwork\B1\V0\BZ1\M-
↪1-0-1'+dotResolution+'.MX1'))
>>> mx=Mx(mx1File=mx1File,NoH5Read=True,NoMxsRead=True)
>>> # ---
>>> # Clean Up LocalHeatingNetwork
>>> # ---
>>> if os.path.exists(mx.h5File):
...    os.remove(mx.h5File)
>>> if os.path.exists(mx.mxsZipFile):
...    os.remove(mx.mxsZipFile)
>>> if os.path.exists(mxsDumpFile):
...    os.remove(mxsDumpFile)
>>> if os.path.exists(mx.h5FileVecs):
...    os.remove(mx.h5FileVecs)
>>> mx.setResultsToMxsFile(maxRecords=1)
1
>>> print(mx._getDfAsOneString())
                         KNOT~~~5356267303828212700~PH KNOT~~~5397990465339071638~QM␣
↪KNOT~~~5736262931552588702~PH
2004-09-22 08:30:00+00:00                                2.0                            0.0␣
↪                          4.1
>>> logger.debug("{:s}: CHANGEHISTORY: {:>10s}: {:>3d}: {:>6s}: {:s}".format('DOCTEST
↪','0.0.41',1,'New',"getMxsVecsFileData"))
>>> timesReq=[]
>>> timesReq.append(mx.df.index[0])
>>> plotTimeDfs=mx.getMxsVecsFileData(timesReq=timesReq)
>>> len(plotTimeDfs)
1
>>> isinstance(plotTimeDfs[0],pd.core.frame.DataFrame)
True
>>> print(mx._getDfVecAsOneString(df=plotTimeDfs[0],regex='KNOT~\S*~\S*~\S*~T$'))
                         KNOT~~~~T
2004-09-22 08:30:00+00:00      60.00
>>> timesReq[0]=timesReq[0]-pd.to_timedelta('1 second')
```

(continues on next page)

(continued from previous page)

```
>>> plotTimeDfs=mx.getMxsVecsFileData(timesReq=timesReq)
>>> len(plotTimeDfs)
0
>>> # ---
>>> # Clean Up LocalHeatingNetwork
>>> # ---
>>> if os.path.exists(mx.h5File):
...     os.remove(mx.h5File)
>>> if os.path.exists(mx.mxsZipFile):
...     os.remove(mx.mxsZipFile)
>>> if os.path.exists(mxsDumpFile):
...     os.remove(mxsDumpFile)
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> # ---
>>> # TinyWDN
>>> # ---
>>> mx1File=os.path.join(path,os.path.join(testDir,'WDTinyWDN\B1\V0\BZ1\M-1-0-1
↪'+dotResolution+'.MX1'))
>>> mx=Mx(mx1File=mx1File,NoH5Read=True,NoMxsRead=True)
>>> print(mx._getMx1DfAsOneString().replace('\n','\n '))
ATTRTYPE  DATALENGTH DATATYPE  DATATYPELENGTH  FLAGS OBJTYPE           OBJTYPE_PK␣
↪TITLE    UNIT
      PH           4     REAL               4      1    KNOT  4711309381204507891    ␣
↪    [bar]
      PH           4     REAL               4      1    KNOT  5179406559406617933    ␣
↪    [bar]
      QM           4     REAL               4      1    KNOT  5179406559406617933    ␣
↪   [m3/h]
>>> print("'''{:s}'''".format(repr(mx.mx2Df).replace('\n','\n   ')))
'''        AttrType                                                                ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                          Data  DataLength DataType ␣
↪DataTypeLength  NOfItems       ObjType
   0   tk                                                                          ␣
↪                                          [5179406559406617933, 5706345341889312301,␣
↪5028754475676510796, 4880261452311588026, 4711309381204507891, 5697271655044179265,␣
↪5165939128645634755, 4790715298669926433, 5629305599838467353, 5388350113283448399,␣
↪4702108116010765829, 5095105260880965618, 5113971272348625691, 5042575626021291052,␣
↪4780213881308610359, 4832703654265095420, 5498009282312522569]         340     CHAR␣
↪             20        17  KNOT
   1   pk                                                                          ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [4673068329861421062]          20     CHAR ␣
↪            20         1  LFKT
   2   tk                                                                          ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [4914542339545953765]          20     CHAR ␣
↪            20         1  OBEH
```

(continues on next page)

(continued from previous page)

```
  3   pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [5295290952622367198]          20     CHAR ␣
↪            20         1  PHI1
  4   pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [5623478865255556069]          20     CHAR ␣
↪            20         1  PUMD
  5   pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [4728046487162557851]          20     CHAR ␣
↪            20         1  PVAR
  6   pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [5377391575499483345]          20     CHAR ␣
↪            20         1  QVAR
  7   tk            [4820249423461369400, 5508607066407516090, 5095827458323717990,␣
↪5700891034663173564, 5729543164571807746, 5067763789937102564, 4799443678518131207,␣
↪5605831797210762998, 5126294008398890918, 5419686039699182683, 5312551747276023841,␣
↪5296152509037292880, 5670035893444309530, 5433880705192526755, 4648047345314768819,␣
↪4984438795139137900, 5644872080928983958, 5148090523913666712, 5461179577260327606,␣
↪4978978527327130204, 5497762617222653432, 5076321356874807093]         440     CHAR␣
↪             20        22  ROHR
  8   N_OF_POINTS
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                (5,␣
↪6, 5, 3, 4, 3, 3, 3, 3, 3, 3, 3, 3, 2, 5, 4, 3, 3, 4, 3, 2, 2)          88     INT4␣
↪              4        22  ROHR
  9   pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [4955975670134047353]          20     CHAR ␣
↪            20         1  SWVT
  10  pk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [4821755615165519990]          20     CHAR ␣
↪            20         1  TEVT
  11  tk
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                          [5466655470152247657]          20     CHAR ␣
↪            20         1  VENT
```

(continues on next page)

(continued from previous page)

```
   12  pk                                                                          ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                                                                  ␣
↪                                        [5736734929574151957]          20     CHAR ␣
↪            20         1  WEVT        '''
>>> mx.setResultsToMxsFile(maxRecords=1)
1
>>> print(mx._getDfAsOneString())
                         KNOT~~~4711309381204507891~PH KNOT~~~5179406559406617933~PH␣
↪KNOT~~~5179406559406617933~QM
2002-05-22 16:16:16+00:00                          1.0                           3.0␣
↪                        140.0
>>> # ---
>>> # Clean Up Tiny WDN
>>> # ---
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> # ---
>>> # GPipe
>>> # ---
>>> mx1File=os.path.join(path,os.path.join(testDir,'WDGPipe\B1\V0\BZ1\M-1-0-1
↪'+dotResolution+'.MX1'))
>>> mx=Mx(mx1File=mx1File)
>>> plotTimeDfs=mx.getMxsVecsFileData()
>>> len(plotTimeDfs[0]['ROHR~*~*~*~PVEC'].iloc[0]) == mx.mx2Df[mx.mx2Df['AttrType'].
↪str.contains('N_OF_POINTS')].iloc[0].Data[0]
True
>>> plotTimeDfs[0]['ROHR~*~*~*~PVEC'].iloc[0][0]
41.0
>>> # ---
>>> # Clean Up GPipe
>>> # ---
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
```

class Mx.**Mx**(*mx1File*, *NoH5Read=False*, *NoMxsRead=False*)

Reading SIR 3S' MX-Files.

Summary:

- (mx1File): use this to profit from previous reads finalized with ToH5()
- (mx1File,NoH5Read=True): use this for a fresh start with implicit .MXS-File read; finalize with ToH5()
- (mx1File,NoH5Read=True,NoMxsRead=True): use this for a fresh start; call setResultsTo...() explicit; finalize with ToH5()
- note that base.Y.h5-File has to be dunped explicitely with ToH5()
- and base.Y.vec.h5-File is written implicitely while (implicit or explicit calls to) setResultsTo...()
- and is deleted explicitely (mx1File,NoH5Read=True) or implicitely (because it is i.e. to old)

Args:

- mx1File (str): base.MX1-File (an XML-File) (base.Y.MX1-File from 90-10 on)
- **NoH5Read (bool):**

False (default - use this to profit from previous reads finalized with ToH5()):

- **If a base.Y.h5-File**
  - exists
  - and is newer (>) than an .MX1-File (base.Y.Mx1-File from 90-10 on)
  - and is newer (>) than an .MXS-File (base.Y.MXS-File from 90-10 on):
    - The base.Y.h5-File is read instead of the .MX1-File.

True (use this for a fresh start):

- An base.Y.h5-File is deleted if existing.
- The .MX1-File is read. (base.Y.Mx1-File from 90-10 on)
- The base.Y.vec.h5-File is newly created in case of an .MXS-File read. (.Y.MXS-File read from 90-10 on)

- **NoMxsRead (bool):**

  True:

  - a .MXS-File is not read (.Y.MXS-File read from 90-10 on)
  - a base.Y.vec.h5-File is not touched

  False (default):

  - **If a .MXS-File (.Y.Mx1-File from 90-10 on)**
    - exists
    - and is newer (>=) than .MX1-File (.Y.Mx1-File from 90-10 on)
    - and base.Y.h5-File is not read:
      - The .MXS-File is read. (.Y.MXS-File is read from 90-10 on)
      - NoH5Read=True will delete base.Y.vec.h5-File.

Attributes:

- **states**
  - h5Read: True, if read from H5
- **fileNames**
  - .mx1File: base.MX1-File (.Y.Mx1-File from 90-10 on)

  derived from mx1File

  - .mx2File: base.MX2-File
  - .mxsFile: base.MXS-File (.Y.MXS-File from 90-10 on)
  - .mxsZipFile base.ZIP
  - .h5File: base.Y.h5-File (.Y.MXS-File from 90-10 on)
  - .h5FileVecs: base.y.vec.h5-File (.Y.MXS-File from 90-10 on)
- .mxRecordStructFmtString
- **dataFrames**
  - .mx1Df

- .mx2Df
- **.df**
  - the .MXS-File(s) Content (.Y.MXS-File from 90-10 on)
  - non Vectordata only
  - index: TIMESTAMP (scenario time)
  - **columns: Values**
    - The following (String-)ID - called Sir3sID - is used as Columnlabel:
    - this Sir3sID consists of ~ separated .MX1-File terms:
    - OBJTYPE~NAME1~NAME2~OBJTYPE_PK~ATTRTYPE
    - Sir3sID regExp Example: (?P<OBJTYPE>S+)~(?P<NAME1>[S]*)~(?P<NAME1>S*)~(?P<OBJTYPE_PK>d+)~(?P<ATTRTYPE>S+)'

Raises: MxError

_initWithMx1()

(Re-)initialize .mx1Df, .mx2Df, .mxRecordStructFmtString and related stuff with .mx1File.

Calls:

- ._parseMx1()
- ._parseMx2()
- ._buildMxRecordStructUnpackFmtString()
- ._buildMxRecordStructUnpackFmtStringPost()

Raises: MxError

_parseMx1()

Parses .mx1File.

Sets

- .mx1Df

Raises: MxError

_parseMx2()

Parses .mx2File.

Sets

- .mx2Df

Raises: MxError

_buildMxRecordStructUnpackFmtString()

(Re-)builds .mxRecordStructFmtString and releated stuff.

Sets

- .mxRecordStructFmtString
- .mx1Df['unpackIdx']

Raises: MxError

_buildMxRecordStructUnpackFmtStringPost()

Stuff todo after buildMxRecordStructUnpackFmtString.

Sets

- .idxCVERSO (idx of CVERSO in MX1)
- .unpackIdxCVERSO (idx of CVERSO in recordData)
- .idxTIMESTAMP (idx of TIMESTAMP in MX1)
- .unpackIdxTIMESTAMP (idx of TIMESTAMP in recordData)
- .mxColumnNames=[] (of non Vectordata without TIMESTAMP in MX1-Sequence)
- .mxColumnNamesVecs=[] (of Vectordata without TIMESTAMP in MX1-Sequence)
- .idxUnpackNonVectorChannels[] (idx in recordData)
- .idxUnpackVectorChannels[] (idx in recordData of the 1st ([0]) Element of the Vector)
- .idxOfNonVectorChannels[] (idx in MX1 without TIMESTAMP)
- .idxVectorChannels[] (idx in MX1)

Raises: MxError

_readMxsFile(*mxsFilePtr*, *mxsVecsH5StorePtr*, *firstTime=None*, *maxRecords=None*)

Args:

- mxsFilePtr: .MXS-File
- mxsVecsH5StorePtr: .vec.h5-File
- **firstTime: used to calculate h5Key for mxsVecsH5Store**
  - None (default): firstTime is set to 1st TIMESTAMP in .MXS-File
  - **else:**
    - caller sets firstTime - in general the youngest TIMESTAMP in the df to be extended
- maxRecords

Returns: df: the mxsFile (non Vectordata only) as DataFrame

- index: TIMESTAMP
- df.index.is_unique
  - might be False
  - because of SIR 3S'
  - 1st Time twice (SNAPSHOTTYPE: STAT+TIME) and Last Time triple (SNAPSHOTTYPE: TIME+TMIN/TMAX)

timesWrittenToMxsVecs: the times written to the mxsVecsH5Store

mxsVecsH5StorePtr

- is used to update the mxsVecsH5Store with mxsFile-Content
  - **Key: microseconds from firstTime**
    - can be all negative if firstTime (in general the youngest TIMESTAMP in the df to be extended) is older than the oldest TIMESTAMP in the .MXS-File
  - **dfVecs**
    - index: TIMESTAMP
    - values: Vectordata for the TIMESTAMP

- the Vectordata for a TIMESTAMP is only written when the Key does _not already exist

Raises: MxError

_checkMxsVecsFile (*fullCheck=False*)

Returns (firstTime,lastTime,NOfTimes).

Args:

- **fullCheck (bool)**
  - False (default): only 1st and last h5Keys are read
  - True: all h5Keys are read

Raises: MxError

_handleMxsVecsFileDeletion (*mxsFile*, *newMxsVecsFile=False*)

Handles the deletion of mxsVecsFile.

Args:

- mxsFile
- newMxsVecsFile (default: False)

mxsVecsFile is DELETED! if:

- existing and older than mxsFile
- or newMxsVecsFile is True

Raises: MxError

setResultsToMxsFile (*mxsFile=None*, *add=False*, *NewH5Vec=False*, *maxRecords=None*)

Sets (default) or adds mxsFile-Content to .df.

Args:

- **mxsFile (str)**
  - None (default): .mxsFile is used
- add (bool): default: False: sets df to mxsFile-Content
- NewH5Vec: False (default); if True, an existing mxsVecsFile will be deleted even if it is newer than mxsFile
- maxRecords

Returns:

- timesWrittenToMxsVecs

.df

- index: TIMESTAMP
- self.df.index.is_unique will be True
- because in SIR 3S'
- 1st Time twice (SNAPSHOTTYPE: STAT+TIME) and Last Time triple (SNAPSHOTTYPE: TIME+TMIN/TMAX)
  - **+TIME** is dropped (STAT is used in df)
  - +TMIN/TMAX are dropped (not used in df)

- and because resulting overlapping TIMESTAMPs due to intersections (add=True) are also dropped

.h5FileVecs

- is updated with mxsFile-Content
- **is DELETED! before if existing and**
  - older than mxsFile
  - or newMxsVecsFile

Raises: MxError

```
>>> mx=mxs['GPipes']
>>> mx.df
>>> mx.setResultsToMxsFile(NewH5Vec=True)
1
```

setResultsToMxsZipFile(*mxsZipFile=None*, *add=False*, *NewH5Vec=False*, *maxRecords=None*)

Sets (default) or adds mxsZipFile-Content to .df.

Args:

- **mxsZipFile (str)**
  - None (default): .mxsFile is used
- add (bool): default: False: sets df to mxsZipFile-Content
- NewH5Vec
- maxRecords

Returns:

- timesWrittenToMxsVecsFromZip

Raises: MxError

ToH5(*h5File=None*)

Stores .mx1Df, .mx2Df, .df to h5File.

Args: h5File(str): default: None: self.h5File is used

.h5File

- is !DELETED! before if existing

Keys used

- /MX1
- /MX2
- /MXS

Raises: MxError

FromH5(*h5File=None*)

Sets .mx1Df, .mx2Df, .df to h5File-Content.

Args:

- **h5File(str)**

- None (default): .h5File is used

Keys expected

- /MX1
- /MX2
- /MXS

Raises: MxError

getMxsVecsFileData (*timesReq=None*)

Args:

- **timesReq: List of requested TIMESTAMPs**
  - if None: a single time only, the firstTime, is considered as requested

Returns:

- **List of dfs with mxsVecsFileData**
  - None, if none of the req. TIMESTAMPs was found
- one TIMESTAMP per df
- index: TIMESTAMP

Raises: MxError

dumpInMxsFormat (*mxsDumpFile=None*)

Dumps in MXS-Format to mxsDumpFile (for testing purposes).

Returns:

- **(TimeStampsDumped, TimeStampsFoundInH5)**
  - normally: TimeStampsDumped=TimeStampsFoundInH5
  - if TimesStamps in self.df are manipulated ...
  - ... the H5-Content remains unchanged
  - in effect the H5-Content can be different from self.df-Content ...
  - während in self.df die Zeiten (Index) immer geordnet und voneinander verschieden sind
  - sind beim H5-Content nur die Keys voneinander verschieden
  - um pruefen zu koennen, ob alle Zeiten in self.df im H5-Content auch gefunden wurden, wird TimeStampsFoundInH5 mit ausgegeben
  - gedumped werden immer alle Zeiten aus self.df
  - * fuer jede im H5-Content nicht gefundene Zeit wird das Ergebnis der zuletzt zuvor gefundenen Zeit ausgegeben

Raises: MxError

_getMx1DfAsOneString (*regex='KNOT~\\S*~\\S*~\\d+~[P|Q]{1}[H|M]{1}$'*)

Returns .mx1Df-Content as one String (for Doctest-Purposes).

Raises: MxError

_getDfAsOneString (*regex='KNOT~\\S*~\\S*~\\d+~[P|Q]{1}[H|M]{1}$'*)

Returns .df-Content as one String (for Doctest-Purposes).

Raises: MxError

_getDfVecAsOneString(*df=None*, *regex='KNOT~\\S*~\\S*~\\S*~[P|Q]{1}[H|M]{1}$'*)

Returns dfVec-Content as one String (for Doctest-Purposes).

Raises: MxError

Mx.**getMicrosecondsFromRefTime**(*refTime*, *time*)

Returns time in microseconds since refTime.

Args:

- refTime
- time

Raises: MxError

exception Mx.**MxError**(*value*)

MxError.

CHAPTER

THREE

XM

```
>>> # ---
>>> # Imports
>>> # ---
>>> import os
>>> import logging
>>> logger = logging.getLogger('PT3S.Xm')
>>> # ---
>>> # path
>>> # ---
>>> if __name__ == "__main__":
...   try:
...      dummy=__file__
...      logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ','path =␣
↪os.path.dirname(__file__)'," ."))
...      path = os.path.dirname(__file__)
...   except NameError:
...      logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ","path =
↪'.' because __file__ not defined: ","from Xm import Xm follows ..."))
...      path = '.'
...      from Xm import Xm
... else:
...    logger.debug("{0:s}{1:s}{2:s}{3:s}".format('DOCTEST: Not __main__ Context: ','_
↪__name__: ',__name__,"path = '."))
...    path = '.'
>>> try:
...    from PT3S import Mx
... except ImportError:
...    logger.debug("{0:s}{1:s}".format("DOCTEST: ImportError: from PT3S import Mx: ",
↪"- trying import Mx instead ... maybe pip install -e . is active ..."))
...    import Mx
>>> # ---
>>> # testDir
>>> # ---
>>> # globs={'testDir':'testdata'}
>>> try:
...    dummy= testDir
... except NameError:
...    testDir='testdata'
>>> import pandas as pd
>>> # ---
>>> # Clean Up
>>> # ---
>>> h5File=os.path.join(os.path.join(path,testDir),'OneLPipe.h5')
>>> if os.path.exists(h5File):
```

(continues on next page)

(continued from previous page)

```
...     os.remove(h5File)
>>> # ---
>>> # Init
>>> # ---
>>> xmlFile=os.path.join(os.path.join(path,testDir),'OneLPipe.XML')
>>> xm=Xm(xmlFile=xmlFile)
>>> # ---
>>> # a View
>>> # ---
>>> v='vKNOT'
>>> v in xm.dataFrames
True
>>> isinstance(xm.dataFrames[v],pd.core.frame.DataFrame)
True
>>> # ---
>>> # ToH5
>>> # ---
>>> xm.ToH5()
>>> os.path.exists(xm.h5File)
True
>>> # ---
>>> # force Read H5 instead of Xml
>>> # ---
>>> os.rename(xm.xmlFile,xm.xmlFile+'.blind')
>>> xm=Xm(xmlFile=xmlFile)
>>> os.rename(xm.xmlFile+'.blind',xm.xmlFile)
>>> # ---
>>> vKNOT=xm.dataFrames['vKNOT']
>>> vKNOT[(vKNOT.KTYP.isin(['QKON','PKON'])) & (vKNOT.BESCHREIBUNG.fillna('').str.
↪startswith('Template Element')==False)].shape
(2, 41)
>>> vROHR=xm.dataFrames['vROHR']
>>> vROHR.shape
(1, 74)
>>> isinstance(vROHR['pXCors'],pd.core.series.Series)
True
>>> vROHR['pXCors'][0]
[0.0, 500.0]
>>> vROHR.pYCors[0]
[0.0, 0.0]
>>> # ---
>>> # getWDirModelDirModelName()
>>> # ---
>>> (wDir,modelDir,modelName,mx1File)=xm.getWDirModelDirModelName()
>>> modelName
'M-1-0-1'
>>> # ---
>>> # H5-Deletion if NoH5Read=True
>>> # ---
>>> if os.path.exists(xm.h5File):
...     os.remove(xm.h5File)
>>> xm=Xm(xmlFile=xmlFile)
>>> xm.ToH5()
>>> os.path.exists(xm.h5File)
True
>>> xm=Xm(xmlFile=xmlFile,NoH5Read=True)
>>> os.path.exists(xm.h5File)
```

(continues on next page)

(continued from previous page)

```
False
>>> # ---
>>> # print-Options
>>> # ---
>>> pd.set_option('display.max_columns',None)
>>> pd.set_option('display.max_rows',None)
>>> pd.set_option('display.max_colwidth',666666)
>>> pd.set_option('display.width',666666666)
>>> # ---
>>> # vKNOT
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vKNOT',dropColList=['LFKT_NAME','LF','LF_min
↪','LF_max','PVAR_NAME','PH','PH_min','PH_max','PZON_NAME','FSTF_NAME','STOF_NAME',
↪'GMIX_NAME','UTMP_NAME','2L_NAME','2L_KVR','fkHYDR','fkFQPS']))
  NAME BESCHREIBUNG              IDREFERENZ      CONT CONT_ID  CONT_LFDNR  CONT_VKNO ␣
↪KTYP LFAKT    QM_EIN QVAR_NAME  QM  QM_min  QM_max KVR  TE  TM XKOR YKOR ZKOR      ␣
↪            pk                   tk  pXCor  pYCor
0    I          NaN  3S5642914844465475844  OneLPipe    1001         NaN        NaN ␣
↪QKON     1  176.7146       NaN NaN     NaN     NaN   0 NaN  10  300  600   10 ␣
↪5642914844465475844  5642914844465475844    0.0    0.0
1    K          NaN  3S5289899964753656852  OneLPipe    1001         NaN        NaN ␣
↪PKON     1         0       NaN NaN     NaN     NaN   0 NaN  10  800  600   10 ␣
↪5289899964753656852  5289899964753656852  500.0    0.0
>>> # ---
>>> # vROHR
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vROHR',dropColList=['NAME_i_2L','NAME_k_2L
↪']))
  BESCHREIBUNG              IDREFERENZ BAUJAHR HAL IPLANUNG KENNUNG      L LZU   RAU␣
↪ZAUS ZEIN ZUML JLAMBS LAMBDA0 ASOLL INDSCHALL KVR AUSFALLZEIT DA   DI   DN KT PN␣
↪REHABILITATION REPARATUR  S WSTEIG WTIEFE LTGR_NAME  LTGR_BESCHREIBUNG SICHTBARKEIT␣
↪VERLEGEART DTRO_NAME                          DTRO_BESCHREIBUNG        E fkSTRASSE␣
↪fkSRAT                   pk                   tk IRTRENN LECKSTART LECKEND␣
↪LECKMENGE LECKORT LECKSTATUS QSVB ZVLIMPTNZ KANTENZV      CONT CONT_ID  CONT_LFDNR␣
↪NAME_i KVR_i TM_i XKOR_i YKOR_i ZKOR_i NAME_k KVR_k TM_k XKOR_k YKOR_k ZKOR_k ␣
↪pXCor_i  pYCor_i  pXCor_k  pYCor_k        pXCors      pYCors    pWAYPXCors ␣
↪pWAYPYCors                                WAYP
0          NaN  3S4737064599036143765    2017   0        1       0  10000   0  0.25  ␣
↪ 0    0    0      1   0.025  1000         0   0            0  0  250  250  0  0    ␣
↪        0         0  0      0      0   STDROHR                NaN            1    ␣
↪999999   STDROHR  Standard-Druckrohre mit di = DN (DIN 2402)  2.1E+11        -1    ␣
↪-1  4737064599036143765  4737064599036143765       0         0       0         0   ␣
↪   0          0    0         0        0  OneLPipe    1001         NaN      I     0 ␣
↪ 10    300    600     10      K     0   10    800    600     10      0.0      0.0  ␣
↪ 500.0      0.0  [0.0, 500.0]  [0.0, 0.0]  [0.0, 500.0]  [0.0, 0.0]  [(300.0, 600.
↪0), (800.0, 600.0)]
>>> # ---
>>> # Clean Up
>>> # ---
>>> if os.path.exists(xm.h5File):
...     os.remove(xm.h5File)
>>> # ---
>>> # LocalHeatingNetwork
>>> # ---
>>> xmlFile=os.path.join(os.path.join(path,testDir),'LocalHeatingNetwork.XML')
>>> xm=Xm(xmlFile=xmlFile)
>>> # ---
```

(continues on next page)

(continued from previous page)

```
>>> # vKNOT
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vKNOT',dropColList=['LFKT_NAME','LF','LF_min
↪','LF_max','PVAR_NAME','PH','PH_min','PH_max','PZON_NAME','FSTF_NAME','STOF_NAME',
↪'GMIX_NAME','UTMP_NAME','2L_NAME','2L_KVR','fkHYDR','fkFQPS']))
           NAME                    BESCHREIBUNG IDREFERENZ                                  ␣
↪          CONT CONT_ID CONT_LFDNR CONT_VKNO  KTYP LFAKT QM_EIN QVAR_NAME  QM  QM_
↪min  QM_max KVR  TE  TM     XKOR     YKOR ZKOR                   pk                       ␣
↪  tk   pXCor  pYCor
0        R-K004                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541539  5706361   20  4638663808856251977 ␣
↪4638663808856251977   799.0  152.0
1        V-K002                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541059  5706265   20  4731792362611615619 ␣
↪4731792362611615619   319.0   56.0
2        V-K001                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2540867  5706228   20  4756962427318766791 ␣
↪4756962427318766791   127.0   19.0
3        V-K000                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2540793  5706209   20  4766681917240867943 ␣
↪4766681917240867943    53.0    0.0
4        R-K001                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2540867  5706228   20  4807712987325933680 ␣
↪4807712987325933680   127.0   19.0
5        R-K003                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541457  5706345   20  4891048046264179170 ␣
↪4891048046264179170   717.0  136.0
6        R-K000                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2540793  5706209   20  4979785838440534851 ␣
↪4979785838440534851    53.0    0.0
7        R-K005                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541627  5706363   20  5183147862966701025 ␣
↪5183147862966701025   887.0  154.0
8           R-L                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1      BHKW  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2540740  5706225   20  5356267303828212700 ␣
↪5356267303828212700     0.0   16.0
9        R-K002                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541059  5706265   20  5364712333175450942 ␣
↪5364712333175450942   319.0   56.0
10       V-K004                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541539  5706361   20  5370423799772591808 ␣
↪5370423799772591808   799.0  152.0
11       V-K005                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541627  5706363   20  5444644492819213978 ␣
↪5444644492819213978   887.0  154.0
```

(continues on next page)

(continued from previous page)

```
12       R-K007                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541899  5706325   20  5508992300317633799 ␣
↪5508992300317633799  1159.0  116.0
13       V-K006                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541790  5706338   20  5515313800585145571 ␣
↪5515313800585145571  1050.0  129.0
14       R-K006                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60  2541790  5706338   20  5543326527366090679 ␣
↪5543326527366090679  1050.0  129.0
15       V-K003                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541457  5706345   20  5646671866542823796 ␣
↪5646671866542823796   717.0  136.0
16          V-L                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1      BHKW  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2540740  5706240   20  5736262931552588702 ␣
↪5736262931552588702     0.0   31.0
17       V-K007                            None         -1  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90  2541899  5706325   20  5741235692335544560 ␣
↪5741235692335544560  1159.0  116.0
18           R2                            None         -1                         ␣
↪          BHKW    1002         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60      170       20   20  5002109894154139899 ␣
↪5002109894154139899   170.0   20.0
19          V-1                            None         -1                         ␣
↪          BHKW    1002         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   1  10  90      140      160   20  5049461676240771430 ␣
↪5049461676240771430   140.0  160.0
20           R3                            None         -1                         ␣
↪          BHKW    1002         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60      140       20   20  5219230031772497417 ␣
↪5219230031772497417   140.0   20.0
21  PKON-Knoten  Druckhaltung - 2 bar Ruhedruck         -1                         ␣
↪          BHKW    1002         -1       NaN  PKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  60  60      200       40   20  5397990465339071638 ␣
↪5397990465339071638   200.0   40.0
22          R-1          Anbindung Druckhaltung         -1                         ␣
↪          BHKW    1002         -1       NaN  QKON     1      0       NaN NaN    ␣
↪NaN     NaN   2  10  60      195       20   20  5557222628687032084 ␣
↪5557222628687032084   195.0   20.0
>>> # ---
>>> # vROHR
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vROHR',dropColList=['NAME_i_2L','NAME_k_2L
↪']))
   BESCHREIBUNG IDREFERENZ BAUJAHR HAL IPLANUNG KENNUNG       L LZU  RAU ZAUS ZEIN␣
↪ZUML JLAMBS LAMBDA0 ASOLL INDSCHALL KVR  AUSFALLZEIT     DA     DI   DN     KT  PN ␣
↪REHABILITATION  REPARATUR    S WSTEIG WTIEFE LTGR_NAME           LTGR_BESCHREIBUNG␣
↪SICHTBARKEIT VERLEGEART DTRO_NAME                      DTRO_BESCHREIBUNG        E␣
↪fkSTRASSE fkSRAT                   pk                    tk IRTRENN LECKSTART␣
↪LECKEND LECKMENGE LECKORT LECKSTATUS QSVB ZVLIMPTNZ KANTENZV                     ␣
↪              CONT CONT_ID CONT_LFDNR  NAME_i KVR_i TM_i   XKOR_i   YKOR_i ZKOR_i ␣
↪NAME_k KVR_k TM_k   XKOR_k   YKOR_k ZKOR_k  pXCor_i  pYCor_i  pXCor_k  pYCor_k   ␣
↪      pXCors          pYCors                                pWAYPXCors ␣
↪                  pWAYPYCors                                             ␣
↪                                   WAYP
```

(continues on next page)

(continued from previous page)

```
0          None         -1    None   0        1       0   88.02   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4613782368750024999  4613782368750024999       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K004     2   60  2541539  5706361     20  R-
↪K005     2   60  2541627  5706363     20    799.0    152.0    887.0    154.0
↪[799.0, 887.0]  [152.0, 154.0]    [807.8999999999069, 895.9500000001863]  [140.
↪09999999962747, 142.04999999981374]
↪[(2541547.9, 5706349.1), (2541635.95, 5706351.05)]
1          None         -1    None   0        1       0  405.96   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4614949065966596185  4614949065966596185       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K002     1   90  2541059  5706265     20  V-
↪K003     1   90  2541457  5706345     20    319.0     56.0    717.0    136.0
↪[319.0, 717.0]   [56.0, 136.0]                [319.0, 716.9500000001863]
↪   [56.049999999813735, 136.0]
↪[(2541059.0, 5706265.05), (2541456.95, 5706345.0)]
2          None         -1    None   0        1       0   83.55   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4637102239750163477  4637102239750163477       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K003     2   60  2541457  5706345     20  R-
↪K004     2   60  2541539  5706361     20    717.0    136.0    799.0    152.0
↪[717.0, 799.0]  [136.0, 152.0]    [725.8500000000931, 807.8999999999069]  [124.
↪04999999981374, 140.09999999962747]
↪[(2541465.85, 5706333.05), (2541547.9, 5706349.1)]
3          None         -1    None   0        1       0   88.02   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4713733238627697042  4713733238627697042       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K004     1   90  2541539  5706361     20  V-
↪K005     1   90  2541627  5706363     20    799.0    152.0    887.0    154.0
↪[799.0, 887.0]  [152.0, 154.0]                [799.0, 887.0499999998137]
↪                [152.0, 154.0]
↪[(2541539.0, 5706361.0), (2541627.05, 5706363.0)]
4          None         -1    None   0        1       0  195.53   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4789218195240364437  4789218195240364437       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K001     1   90  2540867  5706228     20  V-
↪K002     1   90  2541059  5706265     20    127.0     19.0    319.0     56.0
↪[127.0, 319.0]    [19.0, 56.0]                            [127.0, 319.0]
↪    [19.0, 56.049999999813735]
↪[(2540867.0, 5706228.0), (2541059.0, 5706265.05)]
5          None         -1    None   0        1       0  109.77   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4945727430885351042  4945727430885351042       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K006     2   60  2541790  5706338     20  R-
↪K007     2   60  2541899  5706325     20   1050.0    129.0   1159.0    116.0  [1050.
↪0, 1159.0]  [129.0, 116.0]  [1058.8500000000931, 1167.8999999999069]
```

(continues on next page)

(continued from previous page)

```
6          None         -1    None   0        1       0    76.4   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4984202422877610920  4984202422877610920       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K000     1   90  2540793  5706209     20  V-
↪K001     1   90  2540867  5706228     20     53.0      0.0    127.0     19.0
↪[53.0, 127.0]     [0.0, 19.0]               [53.049999999813735, 127.0]
↪[-0.049999999813735485, 19.0]
↪[(2540793.05, 5706208.95), (2540867.0, 5706228.0)]
7          None         -1    None   0        1       0   83.55   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5037777106796980248  5037777106796980248       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K003     1   90  2541457  5706345     20  V-
↪K004     1   90  2541539  5706361     20    717.0    136.0    799.0    152.0
↪[717.0, 799.0]  [136.0, 152.0]                [716.9500000001863, 799.0]
↪                [136.0, 152.0]
↪[(2541456.95, 5706345.0), (2541539.0, 5706361.0)]
8          None         -1    None   0        1       0  164.91   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5123819811204259837  5123819811204259837       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K005     1   90  2541627  5706363     20  V-
↪K006     1   90  2541790  5706338     20    887.0    154.0   1050.0    129.0   [887.
↪0, 1050.0]  [154.0, 129.0]   [887.0499999998137, 1049.9500000001863]
↪[154.0, 128.95000000018626]
↪[(2541627.05, 5706363.0), (2541789.95, 5706337.95)]
9          None         -1    None   0        1       0  195.53   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5266224553324203132  5266224553324203132       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K001     2   60  2540867  5706228     20  R-
↪K002     2   60  2541059  5706265     20    127.0     19.0    319.0     56.0
↪[127.0, 319.0]    [19.0, 56.0]  [135.89999999990687, 327.89999999990687]   [7.
↪0499999998137355, 44.09999999962747]
↪  [(2540875.9, 5706216.05), (2541067.9, 5706253.1)]
10         None         -1    None   0        1       0  405.96   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5379365049009065623  5379365049009065623       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K002     2   60  2541059  5706265     20  R-
↪K003     2   60  2541457  5706345     20    319.0     56.0    717.0    136.0
↪[319.0, 717.0]   [56.0, 136.0]   [327.89999999990687, 725.8500000000931]   [44.
↪09999999962747, 124.04999999981374]
↪[(2541067.9, 5706253.1), (2541465.85, 5706333.05)]
11         None         -1    None   0        1       0  164.91   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5611703699850694889  5611703699850694889       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K005     2   60  2541627  5706363     20  R-
↪K006     2   60  2541790  5706338     20    887.0    154.0   1050.0    129.0   [887.
↪0, 1050.0]  [154.0, 129.0]   [895.9500000001863, 1058.8500000000931]
```

(continues on next page)

(continued from previous page)

```
12         None         -1    None   0        1       0  109.77   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5620197984230756681  5620197984230756681       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K006     1   90  2541790  5706338     20  V-
↪K007     1   90  2541899  5706325     20   1050.0    129.0   1159.0    116.0  [1050.
↪0, 1159.0]  [129.0, 116.0]              [1049.9500000001863, 1159.0]  [128.
↪95000000018626, 116.04999999981374]                                              ␣
↪[(2541789.95, 5706337.95), (2541899.0, 5706325.05)]
13         None         -1    None   0        1       0    76.4   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5647213228462830353  5647213228462830353       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K000     2   60  2540793  5706209     20  R-
↪K001     2   60  2540867  5706228     20     53.0      0.0    127.0     19.0
↪[53.0, 127.0]     [0.0, 19.0]  [61.950000000186265, 135.89999999990687]
↪  [-12.0, 7.0499999998137355]
↪[(2540801.95, 5706197.0), (2540875.9, 5706216.05)]
14         None         -1    None   0        1       0   73.42   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  168.3  160.3  150   0.45 NaN
↪        NaN        NaN    4    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4769996343148550485  4769996343148550485       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1     R-L     2   60  2540740  5706225     20  R-
↪K000     2   60  2540793  5706209     20      0.0     16.0     53.0      0.0
↪[0.0, 53.0]     [16.0, 0.0]     [0.0, 24.0, 45.0, 61.950000000186265]
↪ [16.0, 16.0, -12.0, -12.0]  [(2540740.0, 5706225.0), (2540764.0, 5706225.0),␣
↪(2540785.0, 5706197.0), (2540801.95, 5706197.0)]
15         None         -1    None   0        1       0    68.6   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  168.3  160.3  150   0.45 NaN
↪        NaN        NaN    4    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4939422678063487923  4939422678063487923       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1     V-L     1   90  2540740  5706240     20  V-
↪K000     1   90  2540793  5706209     20      0.0     31.0     53.0      0.0
↪[0.0, 53.0]     [31.0, 0.0]           [0.0, 30.0, 53.049999999813735]       [31.0,␣
↪31.0, -0.049999999813735485]                          [(2540740.0, 5706240.0),␣
↪(2540770.0, 5706240.0), (2540793.05, 5706208.95)]
>>> # ---
>>> # vWBLZ
>>> # ---
>>> print("'''{:s}'''".format(repr(xm.dataFrames['vWBLZ']).replace('\n','\n    ')))
'''   AKTIV BESCHREIBUNG IDIM       NAME OBJTYPE                OBJID
↪  pk
   0      1  Wärmebilanz    0      BLNZ1    KNOT  4731792362611615619 ␣
↪5579937562601803472
   1      1  Wärmebilanz    0      BLNZ1    KNOT  5364712333175450942 ␣
↪5579937562601803472
   2      1  Wärmebilanz    0    BLNZ1u5    KNOT  5183147862966701025 ␣
↪5187647097142898375
   3      1  Wärmebilanz    0    BLNZ1u5    KNOT  5444644492819213978 ␣
↪5187647097142898375
```

(continues on next page)

(continued from previous page)

```
   4      1  Wärmebilanz    0    BLNZ1u5    KNOT  4731792362611615619 ␣
↪5187647097142898375
   5      1  Wärmebilanz    0    BLNZ1u5    KNOT  5364712333175450942 ␣
↪5187647097142898375
   6      1  Wärmebilanz    0  BLNZ1u5u7    KNOT  5183147862966701025 ␣
↪4694700216019268978
   7      1  Wärmebilanz    0  BLNZ1u5u7    KNOT  5444644492819213978 ␣
↪4694700216019268978
   8      1  Wärmebilanz    0  BLNZ1u5u7    KNOT  4731792362611615619 ␣
↪4694700216019268978
   9      1  Wärmebilanz    0  BLNZ1u5u7    KNOT  5364712333175450942 ␣
↪4694700216019268978
   10     1  Wärmebilanz    0  BLNZ1u5u7    KNOT  5508992300317633799 ␣
↪4694700216019268978
   11     1  Wärmebilanz    0  BLNZ1u5u7    KNOT  5741235692335544560 ␣
↪4694700216019268978
   12     1  Wärmebilanz    0      BLNZ5    KNOT  5183147862966701025 ␣
↪5581152085151655438
   13     1  Wärmebilanz    0      BLNZ5    KNOT  5444644492819213978 ␣
↪5581152085151655438'''
>>> # ---
>>> # vAGSN
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vAGSN',end=7,dropColList=['nrObjIdTypeInAgsn
↪']))
  LFDNR                                    NAME AKTIV OBJTYPE                OBJID ␣
↪                 pk                    tk  nrObjIdInAgsn  Layer
0     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  4939422678063487923 ␣
↪5252525269080005909  5252525269080005909              1      1
1     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  4984202422877610920 ␣
↪5252525269080005909  5252525269080005909              2      1
2     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  4789218195240364437 ␣
↪5252525269080005909  5252525269080005909              3      1
3     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  4614949065966596185 ␣
↪5252525269080005909  5252525269080005909              4      1
4     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  5037777106796980248 ␣
↪5252525269080005909  5252525269080005909              5      1
5     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  4713733238627697042 ␣
↪5252525269080005909  5252525269080005909              6      1
6     1  Netzdruckdiagramm VL/RL: BHKW - Netzende   101    ROHR  5123819811204259837 ␣
↪5252525269080005909  5252525269080005909              7      1
>>> # ---
>>> # vFWVB
>>> # ---
>>> print("'''{:s}'''".format(repr(xm.dataFrames['vFWVB']).replace('\n','\n    ')))
'''  BESCHREIBUNG IDREFERENZ   W0  LFK  W0LFK  TVL0  TRS0  LFKT      W  W_min  W_max ␣
↪INDTR  TRSK  VTYP  DPHAUS  IMBG  IRFV                  pk                  tk ␣
↪NAME_i KVR_i TM_i   XKOR_i   YKOR_i ZKOR_i  pXCor_i  pYCor_i  NAME_k KVR_k TM_k  ␣
↪XKOR_k   YKOR_k ZKOR_k  pXCor_k  pYCor_k                                    CONT␣
↪CONT_ID CONT_LFDNR                          WBLZ
   0           1         -1  200  0.8  160.0    90    50  LFKT  160.0  160.0  160.0 ␣
↪    1    55    14     0.7     0   0.0  4643800032883366034  4643800032883366034  V-
↪K002     1   90  2541059  5706265     20    319.0     56.0  R-K002     2   60 ␣
↪2541059  5706265     20    319.0     56.0  Nahwärmenetz mit 1000 kW Anschlussleistu␣
↪   1001         -1  [BLNZ1, BLNZ1u5, BLNZ1u5u7]
   1           3         -1  200  1.0  200.0    90    65  LFKT  200.0  200.0  200.0 ␣
↪    1    65    14     0.7     0   0.0  4704603947372595298  4704603947372595298  V-
↪K004     1   90  2541539  5706361     20    799.0    152.0  R-K004     2   60 ␣
↪2541539  5706361     20    799.0    152.0  Nahwärmenetz mit 1000 kW Anschlussleistu␣
↪   1001         -1                                  []
```

(continues on next page)

(continued from previous page)

```
   2            4         -1  200  0.8  160.0    90    60  LFKT  160.0  160.0  160.0 ␣
↪    1    60    14     0.7     0   0.0  5121101823283893406  5121101823283893406  V-
↪K005     1   90  2541627  5706363     20    887.0    154.0  R-K005     2   60 ␣
↪2541627  5706363     20    887.0    154.0  Nahwärmenetz mit 1000 kW Anschlussleistu␣
↪   1001         -1  [BLNZ1u5, BLNZ1u5u7, BLNZ5]
   3            5         -1  200  0.8  160.0    90    55  LFKT  160.0  160.0  160.0 ␣
↪    1    55    14     0.7     0   0.0  5400405917816384862  5400405917816384862  V-
↪K007     1   90  2541899  5706325     20   1159.0    116.0  R-K007     2   60 ␣
↪2541899  5706325     20   1159.0    116.0  Nahwärmenetz mit 1000 kW Anschlussleistu␣
↪   1001         -1                  [BLNZ1u5u7]
   4            2         -1  200  0.6  120.0    90    60  LFKT  120.0  120.0  120.0 ␣
↪    1    62    14     0.7     0   0.0  5695730293103267172  5695730293103267172  V-
↪K003     1   90  2541457  5706345     20    717.0    136.0  R-K003     2   60 ␣
↪2541457  5706345     20    717.0    136.0  Nahwärmenetz mit 1000 kW Anschlussleistu␣
↪   1001         -1                            []'''
>>> # ---
>>> # vLAYR
>>> # ---
>>> # print("'''{:s}'''".format(repr(xm.dataFrames['vLAYR'].sort_values(['LFDNR','NAME
↪','OBJTYPE','OBJID'],ascending=True)).replace('\n','\n    ')))
>>> print(xm._getvXXXXAsOneString(vXXXX='vLAYR'))
   LFDNR           NAME OBJTYPE                OBJID                    pk            ␣
↪      tk  nrObjInGroup  nrObjtypeInGroup
0      1        Vorlauf    FWES  5638756766880678918  5206516471428693478 ␣
↪5206516471428693478             1                  1
1      1        Vorlauf    KNOT  4731792362611615619  5206516471428693478 ␣
↪5206516471428693478             1                  1
2      1        Vorlauf    KNOT  4756962427318766791  5206516471428693478 ␣
↪5206516471428693478             1                  2
3      1        Vorlauf    KNOT  4766681917240867943  5206516471428693478 ␣
↪5206516471428693478             1                  3
4      1        Vorlauf    KNOT  5049461676240771430  5206516471428693478 ␣
↪5206516471428693478             1                  4
5      1        Vorlauf    KNOT  5370423799772591808  5206516471428693478 ␣
↪5206516471428693478             1                  5
6      1        Vorlauf    KNOT  5444644492819213978  5206516471428693478 ␣
↪5206516471428693478             1                  6
7      1        Vorlauf    KNOT  5515313800585145571  5206516471428693478 ␣
↪5206516471428693478             1                  7
8      1        Vorlauf    KNOT  5646671866542823796  5206516471428693478 ␣
↪5206516471428693478             1                  8
9      1        Vorlauf    KNOT  5736262931552588702  5206516471428693478 ␣
↪5206516471428693478             1                  9
10     1        Vorlauf    KNOT  5741235692335544560  5206516471428693478 ␣
↪5206516471428693478             1                 10
11     1        Vorlauf    ROHR  4614949065966596185  5206516471428693478 ␣
↪5206516471428693478             1                  1
12     1        Vorlauf    ROHR  4713733238627697042  5206516471428693478 ␣
↪5206516471428693478             1                  2
13     1        Vorlauf    ROHR  4789218195240364437  5206516471428693478 ␣
↪5206516471428693478             1                  3
14     1        Vorlauf    ROHR  4939422678063487923  5206516471428693478 ␣
↪5206516471428693478             1                  4
15     1        Vorlauf    ROHR  4984202422877610920  5206516471428693478 ␣
↪5206516471428693478             1                  5
16     1        Vorlauf    ROHR  5037777106796980248  5206516471428693478 ␣
↪5206516471428693478             1                  6
```

(continues on next page)

(continued from previous page)

```
17     1        Vorlauf    ROHR  5123819811204259837  5206516471428693478 ␣
↪5206516471428693478             1                  7
18     1        Vorlauf    ROHR  5620197984230756681  5206516471428693478 ␣
↪5206516471428693478             1                  8
19     1        Vorlauf    VENT  4678923650983295610  5206516471428693478 ␣
↪5206516471428693478             1                  1
20     2       Rücklauf    KLAP  4801110583764519435  4693347477612662930 ␣
↪4693347477612662930             1                  1
21     2       Rücklauf    KNOT  4638663808856251977  4693347477612662930 ␣
↪4693347477612662930             1                  1
22     2       Rücklauf    KNOT  4807712987325933680  4693347477612662930 ␣
↪4693347477612662930             1                  2
23     2       Rücklauf    KNOT  4891048046264179170  4693347477612662930 ␣
↪4693347477612662930             1                  3
24     2       Rücklauf    KNOT  4979785838440534851  4693347477612662930 ␣
↪4693347477612662930             1                  4
25     2       Rücklauf    KNOT  5002109894154139899  4693347477612662930 ␣
↪4693347477612662930             1                  5
26     2       Rücklauf    KNOT  5183147862966701025  4693347477612662930 ␣
↪4693347477612662930             1                  6
27     2       Rücklauf    KNOT  5219230031772497417  4693347477612662930 ␣
↪4693347477612662930             1                  7
28     2       Rücklauf    KNOT  5356267303828212700  4693347477612662930 ␣
↪4693347477612662930             1                  8
29     2       Rücklauf    KNOT  5364712333175450942  4693347477612662930 ␣
↪4693347477612662930             1                  9
30     2       Rücklauf    KNOT  5397990465339071638  4693347477612662930 ␣
↪4693347477612662930             1                 10
31     2       Rücklauf    KNOT  5508992300317633799  4693347477612662930 ␣
↪4693347477612662930             1                 11
32     2       Rücklauf    KNOT  5543326527366090679  4693347477612662930 ␣
↪4693347477612662930             1                 12
33     2       Rücklauf    KNOT  5557222628687032084  4693347477612662930 ␣
↪4693347477612662930             1                 13
34     2       Rücklauf    PUMP  5481331875203087055  4693347477612662930 ␣
↪4693347477612662930             1                  1
35     2       Rücklauf    ROHR  4613782368750024999  4693347477612662930 ␣
↪4693347477612662930             1                  1
36     2       Rücklauf    ROHR  4637102239750163477  4693347477612662930 ␣
↪4693347477612662930             1                  2
37     2       Rücklauf    ROHR  4769996343148550485  4693347477612662930 ␣
↪4693347477612662930             1                  3
38     2       Rücklauf    ROHR  4945727430885351042  4693347477612662930 ␣
↪4693347477612662930             1                  4
39     2       Rücklauf    ROHR  5266224553324203132  4693347477612662930 ␣
↪4693347477612662930             1                  5
40     2       Rücklauf    ROHR  5379365049009065623  4693347477612662930 ␣
↪4693347477612662930             1                  6
41     2       Rücklauf    ROHR  5611703699850694889  4693347477612662930 ␣
↪4693347477612662930             1                  7
42     2       Rücklauf    ROHR  5647213228462830353  4693347477612662930 ␣
↪4693347477612662930             1                  8
43     2       Rücklauf    VENT  4897018421024717974  4693347477612662930 ␣
↪4693347477612662930             1                  1
44     2       Rücklauf    VENT  5525310316015533093  4693347477612662930 ␣
↪4693347477612662930             1                  2
45     3  Kundenanlagen    FWVB  4643800032883366034  5003333277973347346 ␣
↪5003333277973347346             1                  1
```

(continues on next page)

(continued from previous page)

```
46     3  Kundenanlagen    FWVB  4704603947372595298  5003333277973347346 ␣
↪5003333277973347346             1                  2
47     3  Kundenanlagen    FWVB  5121101823283893406  5003333277973347346 ␣
↪5003333277973347346             1                  3
48     3  Kundenanlagen    FWVB  5400405917816384862  5003333277973347346 ␣
↪5003333277973347346             1                  4
49     3  Kundenanlagen    FWVB  5695730293103267172  5003333277973347346 ␣
↪5003333277973347346             1                  5
50     4           BHKW    BSYM  5043395081363401573  5555393404073362943 ␣
↪5555393404073362943             1                  1
51     4           BHKW    TEXT  5056836766824229789  5555393404073362943 ␣
↪5555393404073362943             1                  1
52     4           BHKW    TEXT  5329748935118523443  5555393404073362943 ␣
↪5555393404073362943             1                  2
53     5          Texte    ARRW  4664845735864571219  5394410243594912680 ␣
↪5394410243594912680             1                  1
54     5          Texte    ARRW  4902474974831811106  5394410243594912680 ␣
↪5394410243594912680             1                  2
55     5          Texte    ARRW  5026846801782366678  5394410243594912680 ␣
↪5394410243594912680             1                  3
56     5          Texte    ARRW  5688313372729413840  5394410243594912680 ␣
↪5394410243594912680             1                  4
57     5          Texte    NRCV  4681213816714574464  5394410243594912680 ␣
↪5394410243594912680             1                  1
58     5          Texte    NRCV  4857294696992797631  5394410243594912680 ␣
↪5394410243594912680             1                  2
59     5          Texte    NRCV  4914949875368816179  5394410243594912680 ␣
↪5394410243594912680             1                  3
60     5          Texte    NRCV  4946584950744559030  5394410243594912680 ␣
↪5394410243594912680             1                  4
61     5          Texte    NRCV  4968703141722117357  5394410243594912680 ␣
↪5394410243594912680             1                  5
62     5          Texte    NRCV  5091374651838464239  5394410243594912680 ␣
↪5394410243594912680             1                  6
63     5          Texte    NRCV  5097127385155151127  5394410243594912680 ␣
↪5394410243594912680             1                  7
64     5          Texte    NRCV  5179988968597313889  5394410243594912680 ␣
↪5394410243594912680             1                  8
65     5          Texte    NRCV  5281885868749421521  5394410243594912680 ␣
↪5394410243594912680             1                  9
66     5          Texte    NRCV  5410904806390050339  5394410243594912680 ␣
↪5394410243594912680             1                 10
67     5          Texte    NRCV  5476262878682325254  5394410243594912680 ␣
↪5394410243594912680             1                 11
68     5          Texte    NRCV  5557806245003742769  5394410243594912680 ␣
↪5394410243594912680             1                 12
69     5          Texte    RECT  4994817837124479818  5394410243594912680 ␣
↪5394410243594912680             1                  1
70     5          Texte    RPFL  5158870568935841216  5394410243594912680 ␣
↪5394410243594912680             1                  1
71     5          Texte    TEXT  4628671704393700430  5394410243594912680 ␣
↪5394410243594912680             1                  1
72     5          Texte    TEXT  4654104397990769217  5394410243594912680 ␣
↪5394410243594912680             1                  2
73     5          Texte    TEXT  4666644549022031339  5394410243594912680 ␣
↪5394410243594912680             1                  3
74     5          Texte    TEXT  4693143208412077585  5394410243594912680 ␣
↪5394410243594912680             1                  4
```

(continues on next page)

(continued from previous page)

```
75     5          Texte    TEXT  4768731522550494423  5394410243594912680 ␣
↪5394410243594912680             1                  5
76     5          Texte    TEXT  4770844990228490264  5394410243594912680 ␣
↪5394410243594912680             1                  6
77     5          Texte    TEXT  4782197969172967134  5394410243594912680 ␣
↪5394410243594912680             1                  7
78     5          Texte    TEXT  4855692488683645764  5394410243594912680 ␣
↪5394410243594912680             1                  8
79     5          Texte    TEXT  4965628942555351751  5394410243594912680 ␣
↪5394410243594912680             1                  9
80     5          Texte    TEXT  4995961504641886710  5394410243594912680 ␣
↪5394410243594912680             1                 10
81     5          Texte    TEXT  5017907661719368413  5394410243594912680 ␣
↪5394410243594912680             1                 11
82     5          Texte    TEXT  5028052147238787802  5394410243594912680 ␣
↪5394410243594912680             1                 12
83     5          Texte    TEXT  5036153631350515544  5394410243594912680 ␣
↪5394410243594912680             1                 13
84     5          Texte    TEXT  5054433315422452796  5394410243594912680 ␣
↪5394410243594912680             1                 14
85     5          Texte    TEXT  5108336975548011049  5394410243594912680 ␣
↪5394410243594912680             1                 15
86     5          Texte    TEXT  5262441422409836340  5394410243594912680 ␣
↪5394410243594912680             1                 16
87     5          Texte    TEXT  5297832234834839298  5394410243594912680 ␣
↪5394410243594912680             1                 17
88     5          Texte    TEXT  5370727463979416592  5394410243594912680 ␣
↪5394410243594912680             1                 18
89     5          Texte    TEXT  5421223289472778073  5394410243594912680 ␣
↪5394410243594912680             1                 19
90     5          Texte    TEXT  5501963349880613918  5394410243594912680 ␣
↪5394410243594912680             1                 20
91     5          Texte    TEXT  5502619581048467908  5394410243594912680 ␣
↪5394410243594912680             1                 21
92     5          Texte    TEXT  5540395812045688781  5394410243594912680 ␣
↪5394410243594912680             1                 22
93     5          Texte    TEXT  5550982489075668484  5394410243594912680 ␣
↪5394410243594912680             1                 23
94     5          Texte    TEXT  5610916400841895317  5394410243594912680 ␣
↪5394410243594912680             1                 24
95     5          Texte    TEXT  5646820849868629537  5394410243594912680 ␣
↪5394410243594912680             1                 25
96     5          Texte    TEXT  5696590398594231893  5394410243594912680 ␣
↪5394410243594912680             1                 26
97     5          Texte    TEXT  5697088036451277538  5394410243594912680 ␣
↪5394410243594912680             1                 27
>>> # ---
>>> # vGTXT
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vGTXT',sortList=['CONT_ID','pk'],index=False,
↪header=False))
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                          ␣
↪numerische Anzeige:  4614148870174765680  4614148870174765680                ␣
↪    (219.0, -278.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                           ␣
↪Georeferenzpunkt 2  4628671704393700430  4628671704393700430                (1115.
↪9500000001863, -323.0)
```

(continues on next page)

(continued from previous page)

```
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                          ␣
↪    Block  4666644549022031339  4666644549022031339                              (-58.
↪0, -77.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                  ␣
↪numerische Anzeige  4693143208412077585  4693143208412077585                          ␣
↪    (1211.0, -9.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                Knoten␣
↪und Rohre  4995961504641886710  4995961504641886710                              (570.
↪0, -49.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                       ␣
↪Vorlaufstrang  5017907661719368413  5017907661719368413  (358.20699999993667, 220.
↪39499999955297)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                  ␣
↪LocalHeatingNetwork  5028052147238787802  5028052147238787802                          ␣
↪    (1163.0, 536.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1   Tel. 05131 - 4980-0 ; Fax. 05131␣
↪- 4980-15  5054433315422452796  5054433315422452796                          (-230.0,
↪ -1143.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                     Kontrolle: DH-
↪Massenstrom  5100960407865990868  5100960407865990868                              (-
↪60.0, -160.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                  ␣
↪Wärmebilanz: 3 Kunden  5150752151066924202  5150752151066924202                          ␣
↪      (219.0, -318.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1  eMail. info@3SConsult.de ; www.
↪3SConsult.de  5370727463979416592  5370727463979416592                          (-
↪230.0, -1204.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                Differenzdruck VL-/␣
↪RL-Knoten  5502619581048467908  5502619581048467908                              (1211.
↪0, -49.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                       ␣
↪Kundenanlage  5540395812045688781  5540395812045688781  (1131.9500000001863, 283.
↪95000000018626)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                  ␣
↪Fernwärmeverbraucher  5550982489075668484  5550982489075668484                          ␣
↪     (1050.0, 239.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                       ␣
↪Rücklaufstrang  5610916400841895317  5610916400841895317                          ␣
↪  (570.0, -9.0)
Nahwärmenetz mit 1000 kW Anschlussleistu  1001  -1                                Knoten␣
↪und Rohre  5646820849868629537  5646820849868629537  (358.20699999993667, 174.
↪39499999955297)
                                  BHKW  1002  -1                                  ␣
↪Fernwärmeeinspeiser  4654104397990769217  4654104397990769217                          ␣
↪      (115.0, 80.0)
                                  BHKW  1002  -1                                          ␣
↪    Pumpe  4768731522550494423  4768731522550494423                                  ␣
↪(175.0, 25.0)
                                  BHKW  1002  -1                                      ␣
↪Wärmebilanz Netz:  4770844990228490264  4770844990228490264                          ␣
↪    (90.0, 160.0)
                                  BHKW  1002  -1                                        ␣
↪Speicherung  4782197969172967134  4782197969172967134                                ␣
↪(110.0, 140.0)
                                  BHKW  1002  -1                                     ␣
↪Richtungspfeil  4855692488683645764  4855692488683645764                          ␣
↪(220.0, 105.0)
```

(continues on next page)

(continued from previous page)

```
                                    BHKW  1002  -1                                                  ␣
↪ Verluste  4965628942555351751  4965628942555351751                                (110.
↪0, 145.0)
                                    BHKW  1002  -1                            (Element␣
↪verbinden)  5036153631350515544  5036153631350515544                                ␣
↪(150.0, 90.0)
                                    BHKW  1002  -1                     BHKW Modul 1000␣
↪kW therm.  5056836766824229789  5056836766824229789                                  ␣
↪(35.0, 55.0)
                                    BHKW  1002  -1                                        ␣
↪   Ventil  5108336975548011049  5108336975548011049                                 ␣
↪(205.0, 25.0)
                                    BHKW  1002  -1                                        ␣
↪Verbrauch  5262441422409836340  5262441422409836340                                (110.
↪0, 150.0)
                                    BHKW  1002  -1                                         ␣
↪Einspeisung  5297832234834839298  5297832234834839298                                  ␣
↪(110.0, 155.0)
                                    BHKW  1002  -1                                ␣
↪Druckhaltung 2 bar  5329748935118523443  5329748935118523443                          ␣
↪     (180.0, 65.0)
                                    BHKW  1002  -1                                ␣
↪Numerische Anzeige  5421223289472778073  5421223289472778073                          ␣
↪    (190.0, 115.0)
                                    BHKW  1002  -1                                   ␣
↪Verbindungslinie  5501963349880613918  5501963349880613918                           ␣
↪   (150.0, 95.0)
                                    BHKW  1002  -1                                        ␣
↪   (Text)  5696590398594231893  5696590398594231893                                  ␣
↪(35.0, 50.0)
                                    BHKW  1002  -1                                        ␣
↪   Klappe  5697088036451277538  5697088036451277538                                 ␣
↪(145.0, 25.0)
>>> # ---
>>> # vNRCV
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vNRCV',end=14,dropColList=['DPGR','CONT_LFDNR
↪','pk_ROWS'],sortList=['OBJTYPE','fkOBJTYPE','ATTRTYPE','cRefLfdNr']))
   cRefLfdNr                                    CONT CONT_ID OBJTYPE                 ␣
↪fkOBJTYPE ATTRTYPE              tk_ROWS                   pk                   tk   ␣
↪                              pXYLB
0          1                                     BHKW    1002    FWES ␣
↪5638756766880678918        W  5762106696740202356  4857294696992797631 ␣
↪4857294696992797631                          (90.0, 65.0)
1          1                                     BHKW    1002    KNOT ␣
↪5049461676240771430        T  4723443975311885965  5097127385155151127 ␣
↪5097127385155151127                          (90.0, 95.0)
2          1                                     BHKW    1002    KNOT ␣
↪5219230031772497417        T  5602301870151014230  5557806245003742769 ␣
↪5557806245003742769                          (90.0, 35.0)
3          1                                     BHKW    1002    KNOT ␣
↪5356267303828212700       PH  5000989080893535213  4968703141722117357 ␣
↪4968703141722117357                         (220.0, 25.0)
4          1  Nahwärmenetz mit 1000 kW Anschlussleistu    1001    KNOT ␣
↪5397990465339071638       QM  5134531789044068877  5410059595276504750 ␣
↪5410059595276504750                         (91.0, -94.0)
5          2                                     BHKW    1002    KNOT ␣
↪5397990465339071638       QM  5134531789044068877  5357021981944933535 ␣
↪5357021981944933535  (184.999999464624, 57.99999953107601)
```

(continues on next page)

(continued from previous page)

```
6          1                                        BHKW    1002    KNOT ␣
↪5736262931552588702       PH  4754881272083464445  4681213816714574464 ␣
↪4681213816714574464                          (220.0, 85.0)
7          1  Nahwärmenetz mit 1000 kW Anschlussleistu    1001    KNOT ␣
↪5741235692335544560       DP  4949183695502554728  4914949875368816179 ␣
↪4914949875368816179                         (1234.0, 83.0)
8          1                                        BHKW    1002    PUMP ␣
↪5481331875203087055        N  5563842594211689762  5091374651838464239 ␣
↪5091374651838464239                          (170.0, 45.0)
9          1                                        BHKW    1002    VENT ␣
↪4678923650983295610       QM  5126307362398248950  5410904806390050339 ␣
↪5410904806390050339                         (200.0, 110.0)
10         1  Nahwärmenetz mit 1000 kW Anschlussleistu    1001    WBLZ ␣
↪4694700216019268978      WVB  4778244458749966216  4991097791264453745 ␣
↪4991097791264453745                        (354.0, -225.0)
11         1                                        BHKW    1002    WBLZ ␣
↪5262603207038486299      WES  5690691957596882133  5179988968597313889 ␣
↪5179988968597313889                          (90.0, 155.0)
12         1                                        BHKW    1002    WBLZ ␣
↪5262603207038486299    WSPEI  5153847813311339683  4946584950744559030 ␣
↪4946584950744559030                          (90.0, 140.0)
13         1                                        BHKW    1002    WBLZ ␣
↪5262603207038486299      WVB  5214984699859365639  5281885868749421521 ␣
↪5281885868749421521                          (90.0, 150.0)
>>> # ---
>>> # MxSync() - without Mx-Object
>>> # ---
>>> vROHR=xm.dataFrames['vROHR']
>>> vROHR.shape
(16, 74)
>>> 'vNRCV_Mx1' in xm.dataFrames
False
>>> xm.MxSync()
>>> 'vNRCV_Mx1' in xm.dataFrames
True
>>> vROHR.shape
(16, 76)
>>> print(xm._getvXXXXAsOneString(vXXXX='vROHR',dropColList=['NAME_i_2L','NAME_k_2L
↪']))
   BESCHREIBUNG IDREFERENZ BAUJAHR HAL IPLANUNG KENNUNG       L LZU  RAU ZAUS ZEIN␣
↪ZUML JLAMBS LAMBDA0 ASOLL INDSCHALL KVR  AUSFALLZEIT     DA     DI   DN     KT  PN ␣
↪REHABILITATION  REPARATUR    S WSTEIG WTIEFE LTGR_NAME            LTGR_BESCHREIBUNG␣
↪SICHTBARKEIT VERLEGEART DTRO_NAME                        DTRO_BESCHREIBUNG        E␣
↪fkSTRASSE fkSRAT                   pk                   tk IRTRENN LECKSTART␣
↪LECKEND LECKMENGE LECKORT LECKSTATUS QSVB ZVLIMPTNZ KANTENZV                      ␣
↪              CONT CONT_ID CONT_LFDNR  NAME_i KVR_i TM_i   XKOR_i   YKOR_i ZKOR_i ␣
↪NAME_k KVR_k TM_k   XKOR_k   YKOR_k ZKOR_k  pXCor_i  pYCor_i  pXCor_k  pYCor_k     ␣
↪      pXCors          pYCors                                  pWAYPXCors          ␣
↪                  pWAYPYCors                                                      ␣
↪                                        WAYP  mx2NofPts  mx2Idx
0          None         -1    None   0        1       0   88.02   0  0.1    0    0 ␣
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN  ␣
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe  ␣
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11  ␣
↪    -1     -1  4613782368750024999  4613782368750024999       0         0       0 ␣
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K004     2   60  2541539  5706361     20  R-
↪K005     2   60  2541627  5706363     20    799.0    152.0    887.0    154.0 ␣
↪[799.0, 887.0]  [152.0, 154.0]    [807.8999999999069, 895.9500000001863]  [140.
↪09999999962747, 142.04999999981374]
↪[(2541547.9, 5706349.1), (2541635.95, 5706351.05)]          2       0
```

(continues on next page)

(continued from previous page)

```
1          None         -1    None   0        1       0  405.96   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4614949065966596185  4614949065966596185       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K002     1   90  2541059  5706265     20  V-
↪K003     1   90  2541457  5706345     20    319.0     56.0    717.0    136.0
↪[319.0, 717.0]   [56.0, 136.0]                [319.0, 716.9500000001863]
↪   [56.049999999813735, 136.0]
↪[(2541059.0, 5706265.05), (2541456.95, 5706345.0)]          2       1
2          None         -1    None   0        1       0   83.55   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4637102239750163477  4637102239750163477       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K003     2   60  2541457  5706345     20  R-
↪K004     2   60  2541539  5706361     20    717.0    136.0    799.0    152.0
↪[717.0, 799.0]  [136.0, 152.0]    [725.8500000000931, 807.8999999999069]  [124.
↪04999999981374, 140.09999999962747]
↪[(2541465.85, 5706333.05), (2541547.9, 5706349.1)]          2       2
3          None         -1    None   0        1       0   88.02   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4713733238627697042  4713733238627697042       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K004     1   90  2541539  5706361     20  V-
↪K005     1   90  2541627  5706363     20    799.0    152.0    887.0    154.0
↪[799.0, 887.0]  [152.0, 154.0]                [799.0, 887.0499999998137]
↪                [152.0, 154.0]
↪[(2541539.0, 5706361.0), (2541627.05, 5706363.0)]          2       3
4          None         -1    None   0        1       0  195.53   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4789218195240364437  4789218195240364437       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K001     1   90  2540867  5706228     20  V-
↪K002     1   90  2541059  5706265     20    127.0     19.0    319.0     56.0
↪[127.0, 319.0]    [19.0, 56.0]                              [127.0, 319.0]
↪    [19.0, 56.049999999813735]
↪[(2540867.0, 5706228.0), (2541059.0, 5706265.05)]          2       5
5          None         -1    None   0        1       0  109.77   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4945727430885351042  4945727430885351042       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K006     2   60  2541790  5706338     20  R-
↪K007     2   60  2541899  5706325     20   1050.0    129.0   1159.0    116.0  [1050.
↪0, 1159.0]  [129.0, 116.0]  [1058.8500000000931, 1167.8999999999069]
↪[117.0, 104.09999999962747]
↪[(2541798.85, 5706326.0), (2541907.9, 5706313.1)]          2       7
6          None         -1    None   0        1       0    76.4   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4984202422877610920  4984202422877610920       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K000     1   90  2540793  5706209     20  V-
↪K001     1   90  2540867  5706228     20     53.0      0.0    127.0     19.0
↪[53.0, 127.0]     [0.0, 19.0]                [53.049999999813735, 127.0]
```

(continues on next page)

(continued from previous page)

```
7          None         -1    None   0        1       0   83.55   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5037777106796980248  5037777106796980248       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K003     1   90  2541457  5706345     20  V-
↪K004     1   90  2541539  5706361     20    717.0    136.0    799.0    152.0
↪[717.0, 799.0]  [136.0, 152.0]                [716.9500000001863, 799.0]
↪                [136.0, 152.0]
↪[(2541456.95, 5706345.0), (2541539.0, 5706361.0)]          2       9
8          None         -1    None   0        1       0  164.91   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5123819811204259837  5123819811204259837       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K005     1   90  2541627  5706363     20  V-
↪K006     1   90  2541790  5706338     20    887.0    154.0   1050.0    129.0   [887.
↪0, 1050.0]  [154.0, 129.0]   [887.0499999998137, 1049.9500000001863]
↪[154.0, 128.95000000018626]
↪[(2541627.05, 5706363.0), (2541789.95, 5706337.95)]          2      10
9          None         -1    None   0        1       0  195.53   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5266224553324203132  5266224553324203132       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K001     2   60  2540867  5706228     20  R-
↪K002     2   60  2541059  5706265     20    127.0     19.0    319.0     56.0
↪[127.0, 319.0]    [19.0, 56.0]  [135.89999999990687, 327.89999999990687]   [7.
↪0499999998137355, 44.09999999962747]
↪  [(2540875.9, 5706216.05), (2541067.9, 5706253.1)]          2      11
10         None         -1    None   0        1       0  405.96   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5379365049009065623  5379365049009065623       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K002     2   60  2541059  5706265     20  R-
↪K003     2   60  2541457  5706345     20    319.0     56.0    717.0    136.0
↪[319.0, 717.0]   [56.0, 136.0]   [327.89999999990687, 725.8500000000931]   [44.
↪09999999962747, 124.04999999981374]
↪[(2541067.9, 5706253.1), (2541465.85, 5706333.05)]          2      12
11         None         -1    None   0        1       0  164.91   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5611703699850694889  5611703699850694889       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K005     2   60  2541627  5706363     20  R-
↪K006     2   60  2541790  5706338     20    887.0    154.0   1050.0    129.0   [887.
↪0, 1050.0]  [154.0, 129.0]   [895.9500000001863, 1058.8500000000931]
↪[142.04999999981374, 117.0]
↪[(2541635.95, 5706351.05), (2541798.85, 5706326.0)]          2      13
12         None         -1    None   0        1       0  109.77   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5620197984230756681  5620197984230756681       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  V-K006     1   90  2541790  5706338     20  V-
↪K007     1   90  2541899  5706325     20   1050.0    129.0   1159.0    116.0  [1050.
↪0, 1159.0]  [129.0, 116.0]              [1049.9500000001863, 1159.0]  [128.
```

(continues on next page)

(continued from previous page)

```
13         None         -1    None   0        1       0    76.4   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  114.3  107.1  100  0.325 NaN
↪        NaN        NaN  3.6    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  5647213228462830353  5647213228462830353       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1  R-K000     2   60  2540793  5706209     20  R-
↪K001     2   60  2540867  5706228     20     53.0      0.0    127.0     19.0
↪[53.0, 127.0]     [0.0, 19.0]  [61.950000000186265, 135.89999999990687]
↪  [-12.0, 7.0499999998137355]
↪[(2540801.95, 5706197.0), (2540875.9, 5706216.05)]          2      15
14         None         -1    None   0        1       0   73.42   0  0.1    0    0
↪0      1   0.025  1000         0   2          NaN  168.3  160.3  150   0.45 NaN
↪        NaN        NaN    4    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4769996343148550485  4769996343148550485       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1     R-L     2   60  2540740  5706225     20  R-
↪K000     2   60  2540793  5706209     20      0.0     16.0     53.0      0.0
↪[0.0, 53.0]     [16.0, 0.0]     [0.0, 24.0, 45.0, 61.950000000186265]
↪ [16.0, 16.0, -12.0, -12.0]  [(2540740.0, 5706225.0), (2540764.0, 5706225.0),␣
↪(2540785.0, 5706197.0), (2540801.95, 5706197.0)]          2       4
15         None         -1    None   0        1       0    68.6   0  0.1    0    0
↪0      1   0.025  1000         0   1          NaN  168.3  160.3  150   0.45 NaN
↪        NaN        NaN    4    NaN    NaN   KUMANRO  Beschreibung Leitungsgruppe
↪        1     999999   KUMANRO  Kunststoffmantelrohr DN20-800 PANISOVIT  2.1E+11
↪    -1     -1  4939422678063487923  4939422678063487923       0         0       0
↪      0       0          0    0       NaN      NaN  Nahwärmenetz mit 1000 kW␣
↪Anschlussleistu    1001         -1     V-L     1   90  2540740  5706240     20  V-
↪K000     1   90  2540793  5706209     20      0.0     31.0     53.0      0.0
↪[0.0, 53.0]     [31.0, 0.0]          [0.0, 30.0, 53.049999999813735]       [31.0,␣
↪31.0, -0.049999999813735485]                          [(2540740.0, 5706240.0),␣
↪(2540770.0, 5706240.0), (2540793.05, 5706208.95)]          2       6
>>> # ---------
>>> # MxSync()
>>> # ---------
>>> xm=Xm(xmlFile=xmlFile)
>>> vROHR=xm.dataFrames['vROHR']
>>> vROHR.shape
(16, 74)
>>> 'vNRCV_Mx1' in xm.dataFrames
False
>>> (wDir,modelDir,modelName,mx1File)=xm.getWDirModelDirModelName()
>>> mx=Mx.Mx(mx1File=mx1File)
>>> xm.MxSync(mx=mx)
>>> vROHR.shape
(16, 76)
>>> 'vNRCV_Mx1' in xm.dataFrames
True
>>> # ---
>>> # vNRCV_Mx1
>>> # ---
>>> import re
>>> f=lambda s: re.match('(\S+)~(\S*)~(\S*)~(\d+)~(\S+)',s).group(1)+'~~~'+re.match(
↪'(\S+)~(\S*)~(\S*)~(\d+)~(\S+)',s).group(4)+'~'+re.match('(\S+)~(\S*)~(\S*)~(\d+)~
↪(\S+)',s).group(5)
>>> print(xm._getvXXXXAsOneString(vXXXX='vNRCV_Mx1',end=14,dropColList=['DPGR','CONT_
↪LFDNR','pk_ROWS'],mapFunc={'Sir3sID':f},sortList=['Sir3sID'],fmtFunc={'Sir3sID':f},
↪index=False,header=False))
```

(continues on next page)

(continued from previous page)

```
FWES~~~5638756766880678918~W  1                                        BHKW  1002  FWES␣
↪ 5638756766880678918      W  5762106696740202356  4857294696992797631 ␣
↪4857294696992797631     (90.0, 65.0)
    KNOT~~~5049461676240771430~T  1                                     BHKW  1002 ␣
↪KNOT  5049461676240771430      T  4723443975311885965  5097127385155151127 ␣
↪5097127385155151127     (90.0, 95.0)
    KNOT~~~5219230031772497417~T  1                                     BHKW  1002 ␣
↪KNOT  5219230031772497417      T  5602301870151014230  5557806245003742769 ␣
↪5557806245003742769     (90.0, 35.0)
   KNOT~~~5356267303828212700~PH  1                                     BHKW  1002 ␣
↪KNOT  5356267303828212700     PH  5000989080893535213  4968703141722117357 ␣
↪4968703141722117357    (220.0, 25.0)
   KNOT~~~5397990465339071638~QM  1  Nahwärmenetz mit 1000 kW Anschlussleistu  1001 ␣
↪KNOT  5397990465339071638     QM  5134531789044068877  5410059595276504750 ␣
↪5410059595276504750    (91.0, -94.0)
   KNOT~~~5736262931552588702~PH  1                                     BHKW  1002 ␣
↪KNOT  5736262931552588702     PH  4754881272083464445  4681213816714574464 ␣
↪4681213816714574464    (220.0, 85.0)
   KNOT~~~5741235692335544560~DP  1  Nahwärmenetz mit 1000 kW Anschlussleistu  1001 ␣
↪KNOT  5741235692335544560     DP  4949183695502554728  4914949875368816179 ␣
↪4914949875368816179   (1234.0, 83.0)
    PUMP~~~5481331875203087055~N  1                                     BHKW  1002 ␣
↪PUMP  5481331875203087055      N  5563842594211689762  5091374651838464239 ␣
↪5091374651838464239    (170.0, 45.0)
   VENT~~~4678923650983295610~QM  1                                     BHKW  1002 ␣
↪VENT  4678923650983295610     QM  5126307362398248950  5410904806390050339 ␣
↪5410904806390050339   (200.0, 110.0)
  WBLZ~~~4694700216019268978~WVB  1  Nahwärmenetz mit 1000 kW Anschlussleistu  1001 ␣
↪WBLZ  4694700216019268978    WVB  4778244458749966216  4991097791264453745 ␣
↪4991097791264453745  (354.0, -225.0)
  WBLZ~~~5262603207038486299~WES  1                                     BHKW  1002 ␣
↪WBLZ  5262603207038486299    WES  5690691957596882133  5179988968597313889 ␣
↪5179988968597313889    (90.0, 155.0)
WBLZ~~~5262603207038486299~WSPEI  1                                     BHKW  1002 ␣
↪WBLZ  5262603207038486299  WSPEI  5153847813311339683  4946584950744559030 ␣
↪4946584950744559030    (90.0, 140.0)
  WBLZ~~~5262603207038486299~WVB  1                                     BHKW  1002 ␣
↪WBLZ  5262603207038486299    WVB  5214984699859365639  5281885868749421521 ␣
↪5281885868749421521    (90.0, 150.0)
WBLZ~~~5262603207038486299~WVERL  1                                     BHKW  1002 ␣
↪WBLZ  5262603207038486299  WVERL  4722863010266870887  5476262878682325254 ␣
↪5476262878682325254    (90.0, 145.0)
>>> # ---
>>> # vKNOT
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vKNOT',end=1,dropColList=['LFKT_NAME','LF',
↪'LF_min','LF_max','PVAR_NAME','PH','PH_min','PH_max','PZON_NAME','FSTF_NAME','STOF_
↪NAME','GMIX_NAME','UTMP_NAME','2L_NAME','2L_KVR','fkHYDR','fkFQPS']))
     NAME BESCHREIBUNG IDREFERENZ                                    CONT CONT_ID␣
↪CONT_LFDNR CONT_VKNO  KTYP LFAKT QM_EIN QVAR_NAME  QM  QM_min  QM_max KVR  TE  TM  ␣
↪  XKOR     YKOR ZKOR                  pk                  tk  pXCor  pYCor  mx2Idx
0  R-K004         None         -1  Nahwärmenetz mit 1000 kW Anschlussleistu    1001  ␣
↪      -1       NaN  QKON     1      0       NaN NaN     NaN     NaN   2  10  60 ␣
↪2541539  5706361   20  4638663808856251977  4638663808856251977  799.0  152.0      ␣
↪0
>>> print(xm._getvXXXXAsOneString(vXXXX='vFWVB',dropColList=['BESCHREIBUNG',
↪'IDREFERENZ','W0','W','IRFV','LFK','TVL0','TRS0','LFKT','W_min','W_max','INDTR',
↪'TRSK','VTYP','DPHAUS','IMBG', 'pk','tk','KVR_i','TM_i', 'XKOR_i','YKOR_i', 'ZKOR_i',
↪'pXCor_i','pYCor_i','KVR_k','TM_k', 'XKOR_k','YKOR_k','ZKOR_k','pXCor_k', 'pYCor_k',
↪'CONT','CONT_ID', 'CONT_LFDNR']))
```

(continues on next page)

(continued from previous page)

```
   W0LFK  NAME_i  NAME_k                          WBLZ  mx2Idx
0  160.0  V-K002  R-K002  [BLNZ1, BLNZ1u5, BLNZ1u5u7]       0
1  200.0  V-K004  R-K004                            []       1
2  160.0  V-K005  R-K005  [BLNZ1u5, BLNZ1u5u7, BLNZ5]       2
3  160.0  V-K007  R-K007                   [BLNZ1u5u7]       3
4  120.0  V-K003  R-K003                            []       4
>>> # ---
>>> # vXXXX
>>> # ---
>>> xm.dataFrames['vVBEL_forTestOnly']=xm.dataFrames['vVBEL'].reset_
↪index(inplace=False) # Multiindex to Cols
>>> print(xm._getvXXXXAsOneString(vXXXX='vVBEL_forTestOnly',index=True,dropColList=[
↪'Z_i','pk_i','CONT_i','CONT_VKNO_i','Z_k','pk_k','CONT_k','CONT_VKNO_k','IDREFERENZ
↪','tk']))
   OBJTYPE                OBJID                   BESCHREIBUNG       NAME_i  NAME_k ␣
↪          LAYR       L      D  mx2Idx
0     FWES  5638756766880678918  BHKW - Modul - 1000 kW therm.           R3     V-1 ␣
↪     [Vorlauf]       0     80       0
1     FWVB  4643800032883366034                                1       V-K002  R-K002 ␣
↪[Kundenanlagen]       0    NaN       0
2     FWVB  4704603947372595298                                3       V-K004  R-K004 ␣
↪[Kundenanlagen]       0    NaN       1
3     FWVB  5121101823283893406                                4       V-K005  R-K005 ␣
↪[Kundenanlagen]       0    NaN       2
4     FWVB  5400405917816384862                                5       V-K007  R-K007 ␣
↪[Kundenanlagen]       0    NaN       3
5     FWVB  5695730293103267172                                2       V-K003  R-K003 ␣
↪[Kundenanlagen]       0    NaN       4
6     KLAP  4801110583764519435                             None           R2      R3 ␣
↪    [Rücklauf]       0     80       0
7     PGRP  4986517622672493603                     Pumpengruppe          R-1      R3 ␣
↪            []       0    NaN       0
8     PUMP  5481331875203087055                      Umwälzpumpe          R-1      R2 ␣
↪    [Rücklauf]       0    NaN       0
9     ROHR  4613782368750024999                             None       R-K004  R-K005 ␣
↪    [Rücklauf]   88.02  107.1       0
10    ROHR  4614949065966596185                             None       V-K002  V-K003 ␣
↪     [Vorlauf]  405.96  107.1       1
11    ROHR  4637102239750163477                             None       R-K003  R-K004 ␣
↪    [Rücklauf]   83.55  107.1       2
12    ROHR  4713733238627697042                             None       V-K004  V-K005 ␣
↪     [Vorlauf]   88.02  107.1       3
13    ROHR  4769996343148550485                             None          R-L  R-K000 ␣
↪    [Rücklauf]   73.42  160.3       4
14    ROHR  4789218195240364437                             None       V-K001  V-K002 ␣
↪     [Vorlauf]  195.53  107.1       5
15    ROHR  4939422678063487923                             None          V-L  V-K000 ␣
↪     [Vorlauf]    68.6  160.3       6
16    ROHR  4945727430885351042                             None       R-K006  R-K007 ␣
↪    [Rücklauf]  109.77  107.1       7
17    ROHR  4984202422877610920                             None       V-K000  V-K001 ␣
↪     [Vorlauf]    76.4  107.1       8
18    ROHR  5037777106796980248                             None       V-K003  V-K004 ␣
↪     [Vorlauf]   83.55  107.1       9
19    ROHR  5123819811204259837                             None       V-K005  V-K006 ␣
↪     [Vorlauf]  164.91  107.1      10
20    ROHR  5266224553324203132                             None       R-K001  R-K002 ␣
↪    [Rücklauf]  195.53  107.1      11
```

(continues on next page)

(continued from previous page)

```
21    ROHR  5379365049009065623                          None       R-K002  R-K003  ␣
↪    [Rücklauf]  405.96  107.1      12
22    ROHR  5611703699850694889                          None       R-K005  R-K006  ␣
↪    [Rücklauf]  164.91  107.1      13
23    ROHR  5620197984230756681                          None       V-K006  V-K007  ␣
↪     [Vorlauf]  109.77  107.1      14
24    ROHR  5647213228462830353                          None       R-K000  R-K001  ␣
↪    [Rücklauf]    76.4  107.1      15
25    VENT  4678923650983295610                          None          V-1     V-L  ␣
↪     [Vorlauf]       0    150       0
26    VENT  4897018421024717974                          None          R-L     R-1  ␣
↪    [Rücklauf]       0    150       1
27    VENT  5525310316015533093                          None  PKON-Knoten     R-1  ␣
↪    [Rücklauf]       0     50       2
>>> # ---
>>> # vRART
>>> # ---
>>> print(xm._getvXXXXAsOneString(vXXXX='vRART',index=True,sortList=['INDSTD','NAME
↪']))
  NAME              BESCHREIBUNG                                  INDSTD_TXT  INDSTD␣
↪  DWDT WSOSTD                  pk NAME_KREF1 NAME_KREF2 NAME_SWVT
0   dp  Bezeichnung Regelungsart  Differenzdruck Druckseite, Sollwert Tabelle      55␣
↪ 1E+20      0  5552938346422332788     V-K007     R-K007      SWVT
>>> # ------
>>> # MxAdd
>>> # ------
>>> if 'vNRCV_Mx1' in xm.dataFrames:
...     del xm.dataFrames['vNRCV_Mx1'] # delete MxSync-Result to force MxSync-Call in␣
↪MxAdd
>>> oldShape=xm.dataFrames['vKNOT'].shape
>>> xm.MxAdd()
>>> firstShape=xm.dataFrames['vKNOT'].shape
>>> oldShape[1]<firstShape[1]
True
>>> xm.MxAdd(mx=mx)
>>> secondShape=xm.dataFrames['vKNOT'].shape
>>> secondShape==firstShape
True
>>> xm.MxAdd(mx=mx)
>>> thirdShape=xm.dataFrames['vKNOT'].shape
>>> thirdShape==firstShape
True
>>> xm.dataFrames['vKNOT_forTestOnly']=xm.dataFrames['vKNOT'].rename(columns={'KNOT~*~
↪*~*~PH':'Druck'})
>>> if 'Druck' not in xm.dataFrames['vKNOT_forTestOnly']:
...     xm.dataFrames['vKNOT_forTestOnly'].rename(columns={'KNOT~*~*~*~H':'Druck'},
↪inplace=True)
>>> if 'Druck' not in xm.dataFrames['vKNOT_forTestOnly']:
...     xm.dataFrames['vKNOT_forTestOnly'].rename(columns={'KNOT~*~~*~PH':'Druck'},
↪inplace=True) #09
>>> f = lambda x: round(x,1) if x != None else None
>>> print(xm._getvXXXXAsOneString(vXXXX='vKNOT_forTestOnly',filterColList=['mx2Idx',
↪'KVR','NAME','Druck'],mapFunc={'Druck':f},index=True))
    mx2Idx KVR         NAME  Druck
0        0   2       R-K004    2.3
1        1   1       V-K002    4.0
2        2   1       V-K001    4.1
```

(continues on next page)

(continued from previous page)

```
3        3   1       V-K000    4.1
4        4   2       R-K001    2.0
5        5   2       R-K003    2.3
6        6   2       R-K000    2.0
7        9   2       R-K005    2.3
8       11   2          R-L    2.0
9       12   2       R-K002    2.1
10      13   1       V-K004    3.8
11      15   1       V-K005    3.8
12      16   2       R-K007    2.3
13      17   1       V-K006    3.8
14      18   2       R-K006    2.3
15      20   1       V-K003    3.8
16      21   1          V-L    4.1
17      22   1       V-K007    3.8
18       7   2           R2    4.3
19       8   1          V-1    4.1
20      10   2           R3    4.3
21      14   2  PKON-Knoten    2.0
22      19   2          R-1    2.0
>>> print(xm._getvXXXXAsOneString(vXXXX='vROHR',filterColList=['mx2Idx','L','KVR',
↪'NAME_i','NAME_k','ROHR~*~*~*~QMAV'],mapFunc={'ROHR~*~*~*~QMAV':f},sortList=['ROHR~
↪*~*~*~QMAV','NAME_i'],index=True))
    mx2Idx       L KVR  NAME_i  NAME_k  ROHR~*~*~*~QMAV
13      15    76.4   2  R-K000  R-K001            -23.0
9       11  195.53   2  R-K001  R-K002            -23.0
14       4   73.42   2     R-L  R-K000            -23.0
10      12  405.96   2  R-K002  R-K003            -19.1
2        2   83.55   2  R-K003  R-K004            -15.4
0        0   88.02   2  R-K004  R-K005             -8.5
11      13  164.91   2  R-K005  R-K006             -3.9
5        7  109.77   2  R-K006  R-K007             -3.9
8       10  164.91   1  V-K005  V-K006              3.9
12      14  109.77   1  V-K006  V-K007              3.9
3        3   88.02   1  V-K004  V-K005              8.5
7        9   83.55   1  V-K003  V-K004             15.4
1        1  405.96   1  V-K002  V-K003             19.1
6        8    76.4   1  V-K000  V-K001             23.0
4        5  195.53   1  V-K001  V-K002             23.0
15       6    68.6   1     V-L  V-K000             23.0
>>> print(xm._getvXXXXAsOneString(vXXXX='vFWVB',filterColList=['mx2Idx','NAME_i',
↪'NAME_k','FWVB~*~*~*~W'],mapFunc={'FWVB~*~*~*~W':f},sortList=['FWVB~*~*~*~W','NAME_i
↪'],index=True))
   mx2Idx  NAME_i  NAME_k  FWVB~*~*~*~W
4       4  V-K003  R-K003         120.0
0       0  V-K002  R-K002         160.0
2       2  V-K005  R-K005         160.0
3       3  V-K007  R-K007         160.0
1       1  V-K004  R-K004         200.0
>>> xm.dataFrames['vVBEL_forTestOnly2']=xm.dataFrames['vVBEL'].loc[['ROHR','FWVB'],:].
↪reset_index(inplace=False) # Multiindex to Cols
>>> xm.dataFrames['vVBEL_forTestOnly2'].rename(columns={'KNOT~*~*~*~PH_i':'Druck_i'},
↪inplace=True)
>>> if 'Druck_i' not in xm.dataFrames['vVBEL_forTestOnly2']:
...     xm.dataFrames['vVBEL_forTestOnly2'].rename(columns={'KNOT~*~*~*~H_i':'Druck_i
↪'},inplace=True)
>>> if 'Druck_i' not in xm.dataFrames['vVBEL_forTestOnly2']:
```

(continues on next page)

(continued from previous page)

```
...     xm.dataFrames['vVBEL_forTestOnly2'].rename(columns={'KNOT~*~~*~PH_i':'Druck_i
↪'},inplace=True) #09
>>> f = lambda x: round(x,1) if x != None else None
>>> print(xm._getvXXXXAsOneString(vXXXX='vVBEL_forTestOnly2',filterColList=['OBJTYPE',
↪'mx2Idx','L','D','NAME_i','NAME_k','Druck_i','Q'],mapFunc={'Druck_i':f,'Q':f},
↪sortList=['OBJTYPE','NAME_i','Q'],index=True))
   OBJTYPE  mx2Idx       L      D  NAME_i  NAME_k  Druck_i     Q
0     FWVB       0       0    NaN  V-K002  R-K002      4.0   3.9
4     FWVB       4       0    NaN  V-K003  R-K003      3.8   3.7
1     FWVB       1       0    NaN  V-K004  R-K004      3.8   6.9
2     FWVB       2       0    NaN  V-K005  R-K005      3.8   4.6
3     FWVB       3       0    NaN  V-K007  R-K007      3.8   3.9
20    ROHR      15    76.4  107.1  R-K000  R-K001      2.0 -23.0
16    ROHR      11  195.53  107.1  R-K001  R-K002      2.0 -23.0
17    ROHR      12  405.96  107.1  R-K002  R-K003      2.1 -19.1
7     ROHR       2   83.55  107.1  R-K003  R-K004      2.3 -15.4
5     ROHR       0   88.02  107.1  R-K004  R-K005      2.3  -8.5
18    ROHR      13  164.91  107.1  R-K005  R-K006      2.3  -3.9
12    ROHR       7  109.77  107.1  R-K006  R-K007      2.3  -3.9
9     ROHR       4   73.42  160.3     R-L  R-K000      2.0 -23.0
13    ROHR       8    76.4  107.1  V-K000  V-K001      4.1  23.0
10    ROHR       5  195.53  107.1  V-K001  V-K002      4.1  23.0
6     ROHR       1  405.96  107.1  V-K002  V-K003      4.0  19.1
14    ROHR       9   83.55  107.1  V-K003  V-K004      3.8  15.4
8     ROHR       3   88.02  107.1  V-K004  V-K005      3.8   8.5
15    ROHR      10  164.91  107.1  V-K005  V-K006      3.8   3.9
19    ROHR      14  109.77  107.1  V-K006  V-K007      3.8   3.9
11    ROHR       6    68.6  160.3     V-L  V-K000      4.1  23.0
>>> # ---
>>> # Clean Up LocalHeatingNetwork Xm and Mx
>>> # ---
>>> if os.path.exists(xm.h5File):
...     os.remove(xm.h5File)
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> if os.path.exists(mx.h5File):
...     os.remove(mx.h5File)
>>> # ---
>>> # TinyWDN
>>> # ---
>>> xmlFile=os.path.join(os.path.join(path,testDir),'TinyWDN.XML')
>>> xm=Xm(xmlFile=xmlFile)
>>> # ---
>>> # GPipe
>>> # ---
>>> xmlFile=os.path.join(os.path.join(path,testDir),'GPipe.XML')
>>> xm=Xm(xmlFile=xmlFile)
```

class Xm.**Xm**(*xmlFile*, *NoH5Read=False*)

SIR 3S modelFile to pandas DataFrames.

Args:

- xmlFile (str): SIR 3S modelFile
- **NoH5Read (bool):**

  False (default):

- An existing and newer h5File will be read _instead of xmlFile.
- xmlFile will _not be read (it does even not have to exist)

True:

- An existing h5File will be deleted.
- xmlFile will be read.

Attributes:

- **states**
  - h5Read: True, if read from H5
- xmlFile
- h5File: corresponding h5File(name) derived from xmlFile(name)
- **dataFrames**
  - dict with pandas DataFrames
  - one pandas DataFrame per SIR 3S Objecttype (i.e. KNOT, ROHR, . . .)
  - keys: KNOT, ROHR, . . . and vKNOT, vROHR, . . .
  - **some Views as pandas DataFrames**
    * i.e. vKNOT, vROHR, . . .
    * The Views are designed to deal with tedious groundwork.
    * The Views are aggregated somhwat arbitrary.
    * However: Usage of SIR 3S Modeldata is more convenient and efficient with appropriate Views.
- pXCorZero, pYCorZero

Raises: XmError

_xmlRead()

Reads the SIR 3S modelFile.

- Performs fixes and basic conversions inplace the dataFrames read from modelFile: _convertAndFix()
- Creates some Views: _vXXXX()

Raises: XmError

FromH5(*h5File=None*)

Reads all dataFrames stored in h5File into self.DataFrames.

Args:

h5File:

- (str): the h5File(name) to be read
- (None): self.h5File will be read

- Reads all keys.
- Existing keys in self.dataFrames are overwritten.

Note that after .FromH5() the content of self.dataFrames may differ from the content given by an existing self.xmlFile.

Raises: XmError

ToH5(*h5File=None*)

Stores self.dataFrames to h5File.

Args:

h5File:

- (str): the h5File(name) to be used
- (None): self.h5File will be used

- Stores all keys.
- Existing keys in h5File are overwritten.

Raises: XmError

_convertAndFix()

Performs fixes and basic conversions inplace the dataFrames read from self.xmlFile.

- Fixes and conversions here are integrity-oriented.
- Usage-oriented conversions (i.e. pd.to_numeric and base64.b64decode) are done in the ._vXXXX-methods.

Conversions:

- , > . (converted in: SWVT_ROWT, LFKT_ROWT, QVAR_ROWT, PVAR_ROWT)

Fixes:

- **No SWVT_ROWT, LFKT_ROWT, QVAR_ROWT, PVAR_ROWT?!**
  - * SWVT, LFKT, QVAR, PVAR are constructed to
- 1st Time without Value?! (fixed in: SWVT_ROWT, LFKT_ROWT, QVAR_ROWT, PVAR_ROWT)
- Template Node(s)?!
- **in new Models constructed from SIR 3S**
  - **not all Objectattributes are written?!**
    - * KMOT/TE
    - * FWVB/LFK
    - * LTGR/BESCHREIBUNG
    - * **DTRO_ROWD**
      - · AUSFALLZEIT
      - · PN
      - · REHABILITATION
      - · REPARATUR
      - · WSTEIG, WTIEFE

  - **not all Objecttypes are written?!**
    - CONT

- Models with no PZONs ...
- Models with no GMIXs ...
- Models with no STOFs ...
- empty WBLZ OBJS-BLOBs
- empty LAYR OBJS-BLOBs

Raises: XmError

_constructEmptyDf(*cols=['DummyCol1', 'DummyCol2']*)

Constructs an empty df with cols.

Args:

- cols: list of colNames

Returns: df: constructed df

Raises: XmError

getWDirModelDirModelName()

Returns (wDir,modelDir,modelName,mx1FileName).

Returns: (wDir,modelDir,modelName,mx1FileName)

wDir If wDir as given literally in .xmlFile is not a valid Dir or such a wDir relative to .xmlFile-Path exists the wDir relative is returned.

mx1FileName mx1FileName is assumed to be: .:...WD...B...V...BZ...M... .MX1 If not existing: .*.MX1 (first match is returned) If a suitable mx1File is not existing an INFO-Message is generated.

Raises: XmError

getVersion(*type='BASIS'*)

Returns VERSION-String i.e. Sir3S-90-10.

Args:

- type: BASIS or VARIANTE or BZ; the DATENEBENEn-TYPe from which the VERSION-String is requested

Returns:

- VERSION-String i.e. Sir3S-90-10; Sir3S-90-09 is returned if Attribute VERSION is not available

Raises: XmError

```
>>> xm=xms['OneLPipe']
>>> vStr=xm.getVersion()
>>> import re
>>> m=re.search('Sir(?P<Db3s>[DBdb3Ss]{2})-(?P<Major>\d+)-(?P<Minor>\d+)$',
↪vStr) # i.e. Sir3S-90-10
>>> int(m.group('Major')[0])
9
```

_getvXXXXAsOneString(*vXXXX=None, start=0, end=-1, dropColList=None, filterColList=None, mapFunc={}, sortList=None, fmtFunc={}, index=True, header=True*)

Returns vXXXX-Content as one String (for Doctest-Purposes).

Args:

- vXXXX: df=self.dataFrames[vXXXX]
- start
- end
- dropColList
- filterColList
- mapFunc: col:func: df[col].map(func)
- sortList
- fmtFunc: col:func: passed to df.to_string(formatters=fmtFunc, . . .)
- index
- header

Returns:

- df.to_string(formatters=fmtFunc,index=index,header=header)

Raises: XmError

_vXXXX ( )

Creates all Views.

Views created:

- **BLOB-Data**
  - vLAYR
  - vWBLZ
  - vAGSN u. vAGSN_raw
- **Timeseries**
  - vLFKT
  - vQVAR
  - vPVAR
  - vSWVT
- **Signalmodel**
  - vRSLW
- **Miscellanea**
  - vRART
- **Hydraulicmodel**
  - **Nodes**
    - vVKNO: CONT-Nodes (also called Block-Nodes)
    - vKNOT
    - pXCorZero, pYCorZero
  - **Edges**

        * vROHR: Pipes
        * vFWVB: Housestations (district heating)
    - **all Edges (all; implemented Edges see vVBEL_edges)**
        * vVBEL
- **Annotations**
    - vNRCV
    - vGTXT

Raises: XmError

_vLAYR ()

One row per LAYR and OBJ.

Returns:

columns

LAYR (also called 'Group')

- LFDNR
- NAME

from SIR 3S OBJ BLOB collection:

- OBJTYPE: type (i.e.ROHR) of a LAYR OBJ
- OBJID: pk (or tk?!) of a LAYR OBJ

LAYR IDs

- pk, tk

ANNOTATION

- nrObjInGroup: Element Nr. in LAYR (LFDNR) - should be 1 otherwise the same OBJ occurs in the same LAYR multiple times
- nrObjtypeInGroup: Element Nr. of OBJTYPE in LAYR (LFDNR)

SORTING LFDNR,NAME,OBJTYPE,OBJID

Raises: XmError

_vWBLZ ()

One row per WBLZ and OBJ.

Returns:

columns

WBLZ

- AKTIV
- BESCHREIBUNG
- IDIM
- NAME

from SIR 3S OBJ BLOB collection:

- OBJTYPE: type (always KNOT?!)
- OBJID: pk (or tk?!)

WBLZ IDs

- pk

SORTING NAME,pk

Raises: XmError

_vAGSN ()

One row per AGSN and OBJ.

Returns:

columns

AGSN

- LFDNR
- NAME
- AKTIV
- from SIR 3S OBJ BLOB collection:
  - OBJTYPE: type (i.e.ROHR)
  - OBJID: pk (or tk?!)

AGSN IDs

- pk, tk

Sequence

- **Model**
  - therefore nrObjIdInAgsn (see ANNOTATION below) should be the realwolrd sequence

ANNOTATION

- nrObjIdInAgsn: lfd.Nr. Obj. in AGSN (AGSN is defined by LFDNR not by NAME)
- nrObjIdTypeInAgsn: should be 1
- **Layer** 0=undef bei Netztyp 21: 1=VL, 2=RL, 0=undef wenn keine BN-Trennzeile gefunden wird, wird VL angenommen und gesetzt die BN-Trennzeile wird dem VL (1) zugerechnet

Raises: XmError

_vRART ()

One row per RART.

Returns:

columns:

RART

- NAME
- BESCHREIBUNG
- INDSTD_TXT
- INDSTD (numeric)

- DWDT

RART_BZ

- WSOSTD

ID

- pk

References

- NAME_KREF1
- NAME_KREF2
- NAME_SWVT
- [NAME_RCPL] - only if RCPLs exist

sequence: Model

Raises: XmError

_OBJS (*dfName*)

Decode a column OBJS (a BLOB containing a SIR 3S OBJ collection).

Args: dfName: Name of a dataFrame with column OBJS

columns used (in self.dataFrames[dfName]):

- OBJS (BLOB): i.e.: KNOT~4668229590574507160 . . .
- pk: ID (of the row)
- None is returned if these columns are missing
- in this case no changes in column OBJS in self.dataFrames[dfName]

Returns: column OBJS in self.dataFrames[dfName] is decoded to 'XXXX~' if OBJS was None

dfOBJS: dataFrame with one row per OBJ in OBJS:

columns added (compared to self.dataFrames[dfName]):

- OBJTYPE
- OBJID

rows missing (compared to self.dataFrames[dfName]):

- rows with OBJS None

Raises: XmError

_vLFKT ()

One row per Loadfactor Timeseries.

Returns:

columns

LFKT

- NAME
- BESCHREIBUNG
- INTPOL
- ZEITOPTION

SERIES

- LF: 1st Value
- LF_min
- LF_max

LFKT ID

- pk

Raises: XmError

_vNRCV ( )

One row per NRCV (NumeRiCal Value).

Returns:

columns

ANNOTATIONS

- cRefLfdNr

CONT

- CONT
- CONT_ID
- CONT_LFDNR

DP

Datapointgroup

- DPGR

Datapoint

- OBJTYPE
- fkOBJTYPE
- ATTRTYPE

Datapoint IDs

- pk_ROWS
- tk_ROWS

NRCV IDs

- pk
- tk

PLot Coordinates

- pXYLB: (X,Y): Left,Bottom

Raises: XmError

_vGTXT ( )

One row per GTXT (Graphic TeXT).

Returns:

columns

GTXT

- GRAFTEXT (the text)

CONT

- CONT
- CONT_ID
- CONT_LFDNR

GTXT IDs

- pk
- tk

PLot Coordinates

- pXYLB: (X,Y): Left,Bottom

Raises: XmError

_vSWVT()

One row per Timeseries.

Returns:

columns

SWVT

- NAME
- BESCHREIBUNG
- INTPOL
- ZEITOPTION

SERIES

- W: 1st Value
- W_min
- W_max

SWVT ID

- pk

Raises: XmError

_vRSLW(*vSWVT=None*)

One row per RSLW.

Args:

- vSWVT

Returns:

columns

RSLW

- KA
- BESCHREIBUNG

- INDWBG, INDWNO

RSLW BZ

- INDSLW, SLWKON

CONT

- CONT
- ID

SWVT

- SWVT
- BESCHREIBUNG_SWVT
- INTPOL
- ZEITOPTION

SERIES

- W: 1st Value
- W_min
- W_max

RSLW IDs

- pk
- tk

Raises: XmError

_vQVAR( )

One row per Timeseries.

Returns:

columns

QVAR

- NAME
- BESCHREIBUNG
- INTPOL
- ZEITOPTION

SERIES

- QM: 1st Value
- QM_min
- QM_max

QVAR ID

- pk

Raises: XmError

_vPVAR ( )

One row per Timeseries.

Returns:

columns

PVAR

- NAME
- BESCHREIBUNG
- INTPOL
- ZEITOPTION

SERIES

- PH: 1st Value
- PH_min
- PH_max

QVAR ID

- pk

Raises: XmError

_vVKNO ( )

One row per Blocknode.

Returns:

columns

- NAME
- CONT (Blockname)
- fkKNOT
- fkCONT

Raises: XmError

_vKNOT (*vVKNO=None*, *vQVAR=None*, *vPVAR=None*, *vLFKT=None*)

One row per Node (KNOT).

Args:

- vVKNO
- vQVAR
- vPVAR
- vLFKT

Returns:

rows sequence: Xml

columns

KNOT

- NAME

- BESCHREIBUNG
- IDREFERENZ

CONT

- CONT
- CONT_ID
- CONT_LFDNR
- CONT_VKNO (name of the Block/Container for NAME is a Blocknode)

KNOT

- KTYP
- LFAKT (Umrechnungsfaktor)
- QM_EIN

QVAR

- QVAR_NAME
- QM, QM_min, QM_max

LFKT

- LFKT_NAME
- LF, LF_min, LF_max

PVAR

- PVAR_NAME
- PH, PH_min, PH_max

Zugehoerigkeit

- PZON_NAME
- FSTF_NAME,STOF_NAME,GMIX_NAME
- UTMP_NAME

2L

- 2L_NAME
- 2L_KVR

KNOT

- KVR
- TE
- TM

KNOT

- XKOR, YKOR, ZKOR

KNOT IDs

- pk, tk

KNOT: plot-Coordinates

- pXCor: X-pXCorZero
- pYCor: Y-pYCorZero

Raises: XmError

_vROHR(*vKNOT=None*)

One row per Pipe (ROHR).

Args: vKNOT

Returns:

rows sequence: Xml

columns

ROHR

- BESCHREIBUNG
- IDREFERENZ

ROHR

- BAUJAHR, HAL
- IPLANUNG, KENNUNG

ROHR

- L, LZU, RAU, ZAUS, ZEIN, ZUML
- JLAMBS, LAMBDA0

ROHR

- ASOLL, INDSCHALL

ROHR FW

- NAME_i_2L
- NAME_k_2L
- KVR

DTRO_ROWD

- AUSFALLZEIT, DA , DI , DN , KT , PN , REHABILITATION , REPARATUR , S , WSTEIG , WTIEFE

LTGR

- LTGR_NAME, LTGR_BESCHREIBUNG , SICHTBARKEIT , VERLEGEART

DTRO

- DTRO_NAME, DTRO_BESCHREIBUNG, E

REF

- fkSTRASSE, fkSRAT

ROHR IDs

- pk, tk

ROHR BZ

- ITRENN

- LECKSTART, LECKEND, LECKMENGE, LECKORT, LECKSTATUS

Rest

- QSVB, ZVLIMPTNZ, KANTENZV

CONT

- CONT
- CONT_ID
- CONT_LFDNR

vKNOT

KI

- NAME_i
- KVR_i, TM_i
- XKOR_i, YKOR_i, ZKOR_i

KK

- NAME_k
- KVR_k, TM_k
- XKOR_k, YKOR_k, ZKOR_k

pXCor_i, pYCor_i pXCor_k, pYCor_k

PLOT

- pXCors, pYCors
- pWAYPXCors, pWAYPYCors

Raises: XmError

_vFWVB (*vKNOT=None*, *vLFKT=None*, *vWBLZ=None*)

One row per DistrictHeatingHousestation (FWVB).

Args:

- vKNOT
- vLFKT
- wWBLZ

Returns:

columns

FWVB

- BESCHREIBUNG
- IDREFERENZ
- W0
- LFK
- W0LFK
- TVL0, TRS0

vLFKT

- LFKT
- W, W_min, W_max

FWVB contd.

- INDTR, TRSK
- VTYP
- DPHAUS, IMBG, IRFV

FWVB IDs

- pk, tk

vKNOT

Ki

- NAME_i
- KVR_i, TM_i
- XKOR_i, YKOR_i, ZKOR_i
- pXCor_i, pYCor_i

Kk

- NAME_k
- KVR_k, TM_i
- XKOR_k, YKOR_k, ZKOR_i
- pXCor_k, pYCor_i

vCONT

- CONT
- CONT_ID
- CONT_LFDNR

vWBLZ

- **['BLZ1','BLZ2',...]** list of the WBLZ-Names of the FWVB in alphabetical Order; empty list, if FWVB is not a WBLZ-Member

Raises: XmError

_vVBEL(*vKNOT=None, edges=['ROHR', 'VENT', 'FWVB', 'FWES', 'PUMP', 'KLAP', 'REGV', 'PREG', 'MREG', 'DPRG', 'PGRP'], edgesD=['', 'DN', '', 'DN', '', 'DN', 'DN', 'DN', 'DN', 'DN', ''], indices=['OBJTYPE', 'pk'], mIdxNames=['OBJTYPE', 'OBJID']*)

One row per Edge.

Args:

- vKNOT: df
- edges: list of strs
- edgesD: list of strs
- indices: list of columns which shall be used as MIndex; the columns will be droped; the columns must be delivered by _vVBEL_XXXX

- mIdxNames: list of names for the indices above

Returns: Edge-df returned Edge-df is None if an exception occurs

rows:

- sequence edges: edges
- sequence within edges: Xml

Mindices:

- OBJTYPE: str: 'ROHR','VENT',... [default a MIndex not a column]
- OBJID [default a MIndex not a column]

columns:

- LAYR
- L in m (0 if edge <> ROHR)
- D in mm (NaN if no Diameter could be determined)

columns:

- see _vVBEL_XXXX

_vVBEL_XXXX(*vKNOT=None*, *OBJTYPE=None*)

One row per Edge.

Args:

- vKNOT: df
- **OBJTYPE: str ('ROHR','VENT',...)** self.dataFrames[OBJTYPE] is used to build with vKNOT the returned Edge-df

Returns: Edge-df None is returned if an exception occurs

columns:

- OBJTYPE: str: ROHR,VENT,...
- BESCHREIBUNG
- IDREFERENZ
- pk
- tk
- NAME_i
- CONT_i
- CONT_VKNO_i
- Z_i
- pk_i
- NAME_k
- CONT_k
- CONT_VKNO_k
- Z_k
- pk_k

MxSync (*mx=None*)

Mx: Sir3sID Update in Mx-Object. Xm: NEW 1st Call: vNRCV_Mx1: vNRCV with MX1-Information. Some Xm-Views with MX2-Information (mx2Idx).

Args:

mx: Mx-Object

- If no Mx-Object is given the Mx-Object is constructed.
- Notes:
  - Xm holds no Mx-Object.
  - Method MxSync can be considered as a Sync between Xm and a particular Mx-Object.
  - The Sync has to be done before the 1st MxAdd-Call with the Mx-Object.
  - The Sync-Result is persisted if dfs were read from H5:
    - xm.ToH5() is called if xm.h5Read is True.
    - mx.ToH5() is called (from __Mx1_Sir3sIDUpd) if Sir3sID-Updates occured and mx.h5Read is True.

Raises: XmError

```
>>> xm=xms['LocalHeatingNetwork']
>>> xm.MxSync()
```

MxAdd (*mx=None*, *timeReq=None*)

Add MX-Resultcolumns to some Xm-Views. NEW 1st Call: vROHRVecResults: vNRCV with MX1-Information.

Args:

mx: Mx-Object

- If no Mx-Object is given the Mx-Object is constructed.

timeReq:

- TIMESTAMP
- if None 1st TIME in Mx is used

Views with MX2-Results added:

- in the Xm-Views below col mx2Idx must exist
- mx2Idx is considered to be the last of the Model-cols
- right from mx2Idx Result-cols are added if not already existing
- already existing Result-cols are overwritten
- **mx2Idx-Views:**
  - vKNOT (KNOT...)
  - vROHR (ROHR...) - only Non-VEC-Channel-Results are added
  - vFWVB (FWVB...)
  - vVBEL (KNOT..._i and KNOT..._k and Q)
- NEW 1st Call:
- **vROHRVecResults: VEC-Channel-Results for Pipe-Interior-Pts (IPts):**

  - pk
  - mx2Idx
  - IptIdx: S,0,...,E - Interior Point Index; S=Start EdgeDefNode, E=End EdgeDefNode, 0=1st Ipt in EdgeDefDirection
  - one column per VEC-Channel
- **vAGSN**
  - from vVBEL: KNOT..._i and KNOT..._k and Q
  - from vROHRVecResults: vecResults
  - **Topology:**
    - nextNODE
    - IptIdx
  - **Geometry:**
    - dx
    - x
    - xVbel
    - Z (the corresponding Z_i, Z_k and ZVEC are droped)
  - **Results:**
    - Q: from Q before and QMVEC for PIPEs; in Schnittrichtung; QMVEC is droped
    - **from KNOT...#_i, KNOT...#_k and #VEC:**
      - #1
      - #2
      - ...

      the correspondig 3 columns (2 #KNOT, 1 #VEC) are droped

Notes:

- **The Add-Result is persisted if df were read from H5:**
  - xm.ToH5() is called if xm.h5Read is True.

Raises: XmError

```
>>> xm=xms['LocalHeatingNetwork']
>>> xm.MxSync()
>>> xm.MxAdd()
>>> print(xm._getvXXXXAsOneString(vXXXX='vROHRVecResults',filterColList=[
↪'mx2Idx','IptIdx','ROHR~*~*~*~SVEC'],index=True))
    mx2Idx IptIdx  ROHR~*~*~*~SVEC
0        0      S         0.000000
1        0      E        88.019997
2        1      S         0.000000
3        1      E       405.959991
4        2      S         0.000000
5        2      E        83.550003
6        3      S         0.000000
7        3      E        88.019997
```

(continues on next page)

(continued from previous page)

```
8        5      S         0.000000
9        5      E       195.529999
10       7      S         0.000000
11       7      E       109.769997
12       8      S         0.000000
13       8      E        76.400002
14       9      S         0.000000
15       9      E        83.550003
16      10      S         0.000000
17      10      E       164.910004
18      11      S         0.000000
19      11      E       195.529999
20      12      S         0.000000
21      12      E       405.959991
22      13      S         0.000000
23      13      E       164.910004
24      14      S         0.000000
25      14      E       109.769997
26      15      S         0.000000
27      15      E        76.400002
28       4      S         0.000000
29       4      E        73.419998
30       6      S         0.000000
31       6      E        68.599998
>>> xm=xms['GPipes']
>>> xm.MxSync()
>>> xm.MxAdd()
>>> xm.MxAdd() # test 2nd Call
>>> vAGSN=xm.dataFrames['vAGSN']
>>> schnitt=vAGSN[vAGSN['NAME']=='LR']
>>> xm.dataFrames['schnitt']=schnitt
>>> print(xm._getvXXXXAsOneString(vXXXX='schnitt',filterColList=['OBJTYPE',
↪'NAME_i','NAME_k','IptIdx','nextNODE','x','PH'],index=True))
    OBJTYPE NAME_i NAME_k IptIdx nextNODE         x       PH
79     VENT     GL     G1      S       G1       0.0       40
80     VENT     GL     G1      E       G1       0.0   39.974
81     ROHR     G1    GKS      S      GKS       0.0   39.974
82     ROHR     G1    GKS      0      GKS    5000.0  39.4534
83     ROHR     G1    GKS      1      GKS   10000.0  38.9255
84     ROHR     G1    GKS      2      GKS   15000.0  38.3903
85     ROHR     G1    GKS      3      GKS   20000.0  37.8474
86     ROHR     G1    GKS      4      GKS   25000.0  37.2964
87     ROHR     G1    GKS      5      GKS   30000.0   36.737
88     ROHR     G1    GKS      6      GKS   35000.0  36.1689
89     ROHR     G1    GKS      7      GKS   40000.0  35.5915
90     ROHR     G1    GKS      8      GKS   45000.0  35.0044
91     ROHR     G1    GKS      9      GKS   50000.0  34.4071
92     ROHR     G1    GKS     10      GKS   55000.0  33.7991
93     ROHR     G1    GKS     11      GKS   60000.0  33.1799
94     ROHR     G1    GKS     12      GKS   65000.0  32.5486
95     ROHR     G1    GKS     13      GKS   70000.0  31.9048
96     ROHR     G1    GKS     14      GKS   75000.0  31.2475
97     ROHR     G1    GKS     15      GKS   80000.0  30.5759
98     ROHR     G1    GKS     16      GKS   85000.0  29.8891
99     ROHR     G1    GKS     17      GKS   90000.0  29.1859
100    ROHR     G1    GKS     18      GKS   95000.0  28.4653
101    ROHR     G1    GKS     19      GKS  100000.0  27.7258
```

(continues on next page)

(continued from previous page)

```
102    ROHR     G1    GKS     20      GKS  105000.0  26.9659
103    ROHR     G1    GKS     21      GKS  110000.0  26.1838
104    ROHR     G1    GKS     22      GKS  115000.0  25.3776
105    ROHR     G1    GKS     23      GKS  120000.0  24.5449
106    ROHR     G1    GKS     24      GKS  125000.0  23.6829
107    ROHR     G1    GKS     25      GKS  130000.0  22.7884
108    ROHR     G1    GKS     26      GKS  135000.0  21.8575
109    ROHR     G1    GKS     27      GKS  140000.0  20.8855
110    ROHR     G1    GKS     28      GKS  145000.0  19.8664
111    ROHR     G1    GKS     29      GKS  150000.0  18.7928
112    ROHR     G1    GKS     30      GKS  155000.0  17.6551
113    ROHR     G1    GKS      E      GKS  160000.0  16.4405
114    VENT    GKS    GKD      S      GKD  160000.0  16.4404
115    VENT    GKS    GKD      E      GKD  160000.0  16.3758
116    ROHR    GKD     G3      S       G3  160000.0  16.3758
117    ROHR    GKD     G3      0       G3  165000.0  15.0583
118    ROHR    GKD     G3      E       G3  170000.0  13.6122
119    ROHR     G4     G3      E       G4  170000.0  13.6122
120    ROHR     G4     G3      0       G4  175000.0  11.9873
121    ROHR     G4     G3      S       G4  180000.0  10.1062
122    VENT     G4     GR      S       GR  180000.0  10.1062
123    VENT     G4     GR      E       GR  180000.0       10
```

_Xm__Mx1_Sir3sIDUpd(*mx*)

Update NAME1,2 and Sir3sID in mx.mx1Df and mx.df.

Args: mx: Mx-Object

Notes:

The following Channels are updated:

- KNOT
- WBLZ
- RXXXX
- all Channels in vVBEL

mx.ToH5() is called if Sir3sID-Updates occured and mx.h5Read is True.

Raises: XmError

_Xm__Mx1_Sir3sIDUpd_ObjTypeNode(*mx=None*, *dfUpd=None*, *dfNAME1=None*, *NAME1Col='NAME'*)

Update Sir3sID and NAME1 in mx.mx1Df and mx.df for Channels in dfUpd.

Args: mx: Mx-Object dfUpd: df with OBJTYPE,NAME1,NAME2,OBJTYPE_PK,ATTRTYPE,Sir3sID to be updated dfNAME1: df with NAME1-Information NAME1Col: col in dfNAME1 with NAME1-Information

Returns: nOfSir3sIDsUpdated

Raises: XmError

_Xm__Mx1_vNRCV(*mx*)

vNRCV_Mx1 (vNRCV with Mx1-Information).

Args: mx: Mx-Object

self.dataFrames['vNRCV_Mx1']

index

- reindex

FILTERed

- existing MX-Channels only
- cRefLfdNr: 1st references only

SORTed

- Sir3sID

columns NEW

- Sir3sID

Raises: XmError

_Xm__Mx2_vFWVB(*mx*)

Mx2-Information into vFWVB.

Args: mx: Mx-Object

self.dataFrames['vFWVB']

columns NEW

- mx2Idx

Raises: XmError

_Xm__Mx2_vKNOT(*mx*)

Mx2-Information into vKNOT.

Args: mx: Mx-Object

self.dataFrames['vKNOT']

columns NEW

- mx2Idx

Raises: XmError

_Xm__Mx2_vROHR(*mx*)

Mx2-Information into vROHR.

Args: mx: Mx-Object

self.dataFrames['vROHR']

columns NEW

- mx2NoPts
- mx2Idx

Raises: XmError

_Xm__Mx2_vVBEL(*mx, edges=['ROHR', 'VENT', 'FWVB', 'FWES', 'PUMP', 'KLAP', 'REGV', 'PREG', 'MREG', 'DPRG', 'PGRP']*)

Mx2-Information into vVBEL.

Args: mx: Mx-Object

self.dataFrames['vVBEL']:

columns NEW

- mx2Idx
- **Notes:**
  - for all edges mx2Idx is taken from mx.mx2Df

Raises: XmError

_Xm__MxAddForOneDf (*dfTarget=None*, *dfSource=None*, *multiIndexKey=None*, *testStr='testStr'*)

Add MX2-Resultdata from dfSource as cols to dfTarget.

Args: dfTarget: df with col mx2Idx dfSource: df with mx2Idx-corresponding index and cols (containing MX2-Resultdata) to be added multiIndexKey: value for 1st Index if dfTarget is Multiindexed - i.e. 'XXXX'

Notes:

- all cols from dfSource are added at the end of dfTarget in dfSource-sequence
- the cols can already exist in dfTarget
- if so, _all cols must already exist ...
- ... the dfTarget-sequence should but must be not necessary the dfSource-sequence

Returns: dfTarget

Raises: XmError

CHAPTER

FOUR

RM

```
>>> # ---
>>> # SETUP
>>> # ---
>>> import os
>>> import logging
>>> logger = logging.getLogger('PT3S.Rm')
>>> # ---
>>> # path
>>> # ---
>>> if __name__ == "__main__":
...   try:
...      dummy=__file__
...      logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ','path =␣
↪os.path.dirname(__file__)'," ."))
...      path = os.path.dirname(__file__)
...   except NameError:
...      logger.debug("{0:s}{1:s}{2:s}".format('DOCTEST: __main__ Context: ","path =
↪'.' because __file__ not defined and: "," from Rm import Rm"))
...      path = '.'
...      from Rm import Rm
... else:
...    path = '.'
...    logger.debug("{0:s}{1:s}".format('Not __main__ Context: ","path = '.' ."))
>>> try:
...    from PT3S import Mx
... except ImportError:
...    logger.debug("{0:s}{1:s}".format("DOCTEST: from PT3S import Mx: ImportError: ",
↪"trying import Mx maybe pip install -e . is active ..."))
...    import Mx
>>> try:
...    from PT3S import Xm
... except ImportError:
...    logger.debug("{0:s}{1:s}".format("DOCTEST: from PT3S import Xm: ImportError: ",
↪"trying import Xm maybe pip install -e . is active ..."))
...    import Xm
>>> # ---
>>> # testDir
>>> # ---
>>> # globs={'testDir':'testdata'}
>>> try:
...    dummy= testDir
... except NameError:
...    testDir='testdata'
>>> # ---
```

(continues on next page)

(continued from previous page)

```
>>> # dotResolution
>>> # ---
>>> # globs={'dotResolution':''}
>>> try:
...     dummy= dotResolution
... except NameError:
...     dotResolution=''
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> pd.set_option('display.max_columns',None)
>>> pd.set_option('display.width',666666666)
>>> # ---
>>> # LocalHeatingNetwork SETUP
>>> # ---
>>> xmlFile=os.path.join(os.path.join(path,testDir),'LocalHeatingNetwork.XML')
>>> xm=Xm.Xm(xmlFile=xmlFile)
>>> mx1File=os.path.join(path,os.path.join(testDir,'WDLocalHeatingNetwork\B1\V0\BZ1\M-
↪1-0-1'+dotResolution+'.MX1'))
>>> mx=Mx.Mx(mx1File=mx1File,NoH5Read=True,NoMxsRead=True)
>>> mx.setResultsToMxsFile(NewH5Vec=True)
5
>>> xm.MxSync(mx=mx)
>>> rm=Rm(xm=xm,mx=mx)
>>> # ---
>>> # Plot 3Classes False
>>> # ---
>>> plt.close('all')
>>> ppi=72 # matplotlib default
>>> dpi_screen=2*ppi
>>> fig=plt.figure(dpi=dpi_screen,linewidth=1.)
>>> timeDeltaToT=mx.df.index[2]-mx.df.index[0]
>>> # 3Classes und FixedLimits sind standardmaessig Falsch; RefPerc ist␣
↪standardmaessig Wahr
>>> # die Belegung von MCategory gemaess FixedLimitsHigh/Low erfolgt immer ...
>>> pFWVB=rm.pltNetDHUS(timeDeltaToT=timeDeltaToT,pFWVBMeasureCBFixedLimitHigh=0.80,
↪pFWVBMeasureCBFixedLimitLow=0.66,pFWVBGCategory=['BLNZ1u5u7'],pVICsDf=pd.DataFrame({
↪'Kundenname': ['VIC1'],'Knotenname': ['V-K007']}))
>>> # ---
>>> # Check pFWVB Return
>>> # ---
>>> f=lambda x: "{0:8.5f}".format(x)
>>> print(pFWVB[['Measure','MCategory','GCategory','VIC']].to_string(formatters={
↪'Measure':f}))
   Measure MCategory  GCategory   VIC
0  0.80971       Top  BLNZ1u5u7   NaN
1  0.66692    Middle              NaN
2  0.66043    Middle  BLNZ1u5u7   NaN
3  0.65551    Bottom  BLNZ1u5u7  VIC1
4  0.68548    Middle              NaN
>>> # ---
>>> # Print
>>> # ---
>>> (wD,fileName)=os.path.split(xm.xmlFile)
>>> (base,ext)=os.path.splitext(fileName)
>>> plotFileName=wD+os.path.sep+base+'.'+'pdf'
>>> if os.path.exists(plotFileName):
...     os.remove(plotFileName)
```

(continues on next page)

(continued from previous page)

```
>>> plt.savefig(plotFileName,dpi=2*dpi_screen)
>>> os.path.exists(plotFileName)
True
>>> # ---
>>> # Plot 3Classes True
>>> # ---
>>> plt.close('all')
>>> # FixedLimits wird automatisch auf Wahr gesetzt wenn 3Classes Wahr ...
>>> pFWVB=rm.pltNetDHUS(timeDeltaToT=timeDeltaToT,pFWVBMeasure3Classes=True,
↪pFWVBMeasureCBFixedLimitHigh=0.80,pFWVBMeasureCBFixedLimitLow=0.66)
>>> # ---
>>> # LocalHeatingNetwork Clean Up
>>> # ---
>>> if os.path.exists(mx.h5File):
...     os.remove(mx.h5File)
>>> if os.path.exists(mx.mxsZipFile):
...     os.remove(mx.mxsZipFile)
>>> if os.path.exists(mx.h5FileVecs):
...     os.remove(mx.h5FileVecs)
>>> if os.path.exists(plotFileName):
...     os.remove(plotFileName)
```

Rm.**pltNetFigAx**(*pDf*, ***kwds*)

Erzeugt eine für die Netzdarstellung verzerrungsfreie Axes-Instanz.

- verwendet gcf() (will return an existing figure if one is open, or it will make a new one if there is no active figure)
- an already existing figure might be created this way: fig=plt.figure(dpi=2*72,linewidth=1.)
- errechnet die verzerrungsfreie Darstellung unter Berücksichtigung einer zukünftigen horizontalen Farblegende
- erzeugt eine Axes-Instanz
- setzt Attribute der Axes-Instanz
- setzt Attribute der Figure-Instanz

Args: pDf: dataFrame

Coordinates:

- pXCor_i: colName in pDf (default: 'pXCor_i'): x-Start Coordinate of all Edges to be plotted
- pYCor_i: colName in pDf (default: 'pYCor_i'): y-Start Coordinate of all Edges to be plotted
- pXCor_k: colName in pDf (default: 'pXCor_k'): x-End Coordinate of all Edges to be plotted
- pYCor_k: colName in pDf (default: 'pYCor_k'): y-End Coordinate of all Edges to be plotted

Colorlegend:

- CBFraction: fraction of original axes to use for colorbar (default: 0.05)
- CBHpad: fraction of original axes between colorbar and new image axes (default: 0.0275)

Figure:

- pltTitle: title [not suptitle] (default: 'pltNetFigAx')
- figFrameon: figure frame (background): displayed or invisible (default: True)

- figEdgecolor: edge color of the Figure rectangle (default: 'black')
- figFacecolor: face color of the Figure rectangle (default: 'white')

`Rm.pltNetNodes`(*pDf*, ***kwds*)

Scatters NODEs on gca().

Args: pDf: dataFrame

NODE: Size (Attribute)

- pAttribute: colName (default: 'Attribute') in pDf
- pSizeFactor: (deafault: 1.)
- scatter Sy-Area in pts^2 = pSizeFactor * Attribute

NODE: Color (Measure)

- pMeasure: colName (default: 'Measure') in pDf
- pMeasureColorMap (default: plt.cm.autumn)
- pMeasureAlpha (default: 0.9)
- pMeasureClip (default: False)
- CBFixedLimits (default: True)
- CBFixedLimitLow (default: 0.)
- CBFixedLimitHigh (default: 1.)

NODE: 3Classes

- pMeasure3Classes (default: True)
- pMCategory: colName (default: 'MCategory') in pDf
- pMCatTopTxt (default: 'Top')
- pMCatMidTxt (default: 'Middle')
- pMCatBotTxt (default: 'Bottom')
- pMCatTopColor (default: 'palegreen')
- pMCatTopAlpha (default: 0.9)
- pMCatTopClip (default: False)
- pMCatMidColorMap (default: plt.cm.autumn)
- pMCatMidAlpha (default: 0.9)
- pMCatMidClip (default: False)
- pMCatBotColor (default: 'violet')
- pMCatBotAlpha (default: 0.9)
- pMCatBotClip (default: False)

NODE:

- pXCor: colName (default: 'pXCor_i') in pDf
- pYCor: colName (default: 'pYCor_i') in pDf

Returns: (pcN, vmin, vmax)

- pcN: die mit Farbskala gezeichneten Symbole
- vmin/vmax: die für die Farbskala verwendeten Extremalwerte

`Rm.`**`pltNetPipes`**(*pDf*, ***kwds*)

Plots Lines with Marker on gca().

Args: pDf: dataFrame

PIPE-Line:

- pAttribute: column in pDf (default: 'Attribute')
- pAttributeLs (default: '-')
- pAttributeSizeFactor: plot linewidth in pts = pAttributeSizeFactor (default: 1.0) * Attribute
- pAttributeColorMap (default: plt.cm.binary)
- pAttributeColorMapUsageStart (default: 1./3; Wertebereich: [0,1])
  - Farbskala nach vorh. min./max. Wert
  - die Farbskala wird nur ab UsageStart genutzt
  - d.h. Werte die eine "kleinere" Farbe hätten, bekommen die Farbe von UsageStart

PIPE-Marker:

- pMeasure: column in pDf (default: 'Measure')
- pMeasureMarker (default: '.')
- pMeasureSizeFactor: plot markersize in pts = pMeasureSizeFactor (default: 1.0) * Measure
- pMeasureColorMap (default: plt.cm.cool)
- pMeasureColorMapUsageStart (default: 0.; Wertebereich: [0,1])
  - Farbskala nach vorh. min./max. Wert
  - die Farbskala wird nur ab UsageStart genutzt
  - d.h. Werte die eine "kleinere" Farbe hätten, bekommen die Farbe von UsageStart

PIPE:

- pWAYPXCors: column in pDf (default: 'pWAYPXCors')
- pWAYPYCors: column in pDf (default: 'pWAYPYCors')
- pClip (default: False)

`Rm.`**`pltNetLegendColorbar`**(*pc*, *pDf*, ***kwds*)

Erzeugt eine Axes cax für den Legendenbereich aus ax (=gca()) und zeichnet auf cax die Farblegende (die Farbskala mit allen Eigenschaften).

Args: pc: (eingefaerbte) PathCollection (aus pltNetNodes); wird für die Erzeugung der Farbskala zwingend benoetigt pDf: dataFrame (default: None)

Measure:

- pMeasure: colName in pDf (default: 'Measure')
- pMeasureInPerc: Measure wird interpretiert in Prozent [0-1] (default: True)
- pMeasure3Classes (default: False d.h. Measure wird nicht in 3 Klassen dargestellt)

CBFixedLimits (Ticks):

- CBFixedLimits (default: False d.h. Farbskala nach vorh. min./max. Wert)
- CBFixedLimitLow (default: .10)
- CBFixedLimitHigh (default: .95)

Label:

- pMeasureUNIT (default: '[]')
- pMeasureTYPE (default: '')

CB

- CBFraction: fraction of original axes to use for colorbar (default: 0.05)
- CBHpad: fraction of original axes between colorbar and new image axes (default: 0.0275)
- CBLabelPad (default: -50)
- CBTicklabelsHPad (default: 0.)
- CBAspect: ratio of long to short dimension (default: 10.)
- CBShrink: fraction by which to shrink the colorbar (default: 0.3)
- CBAnchorHorizontal: horizontaler Fußpunkt der colorbar in Plot-% (default: 0.)
- CBAnchorVertical: vertikaler Fußpunkt der colorbar in Plot-% (default: 0.2)

Return: cax

`Rm.`**`pltNetLegendColorbar3Classes`**(*pDf*, ***kwds*)

Zeichnet auf gca() die ergaenzenden Legendeninformationen bei 3 Klassen.

- scatters the Top-Symbol
- scatters the Bot-Symbol
- the "Mid-Symbol" is the (already existing) colorbar with (already existing) ticks and ticklabels

Args:

pDf: dataFrame

Category:

- pMCategory: colName in pDf (default: 'MCategory')
- pMCatTopText
- pMCatMidText
- pMCatBotText

CBLegend (3Classes) - Parameterization of the representative Symbols

- CBLe3cTopVPad (default: 1+1*1/4)
- CBLe3cMidVPad (default: .5)
- CBLe3cBotVPad (default: 0-1*1/4)
  - 1 is the height of the Colorbar
  - the VPads (the vertical Sy-Positions) are defined in cax.transAxes Coordinates
  - cax is the Colorbar Axes
- CBLe3cSySize=10**2 (Sy-Area in pts^2)

- CBLe3cSyType='o'

Color:

- pMCatBotColor='violet'
- pMCatTopColor='palegreen'

Returns: (bbTop, bbMid, bbBot): the boundingBoxes of the 3Classes-Symbols

class Rm.**Rm**(*xm=None*, *mx=None*)

Args: xm: Xm.Xm Object

mx: Mx.Mx Object

pltNetDHUS(***kwds*)

Plot: Net: DistrictHeatingUnderSupply.

Args (optional):

TIMEs (als TIMEDELTA zu Szenariumbeginn):

- timeDeltaToRef: Reference Scenariotime (for MeasureInRefPerc-Calculations) (default: pd.to_timedelta('0 seconds'))
- timeDeltaToT: Scenariotime (default: pd.to_timedelta('0 seconds'))

FWVB

- **pFWVBFilterFunction: Filterfunction to be applied to FWVB to determine the FWVB to be plotted**
  - **default: lambda df: (df.CONT_ID.astype(int).isin([1001])) & (df.W0LFK>0)**
    - **CONT_IDisIn: [1001]**
      - um zu vermeiden, dass FWVB aus Bloecken gezeichnet werden (unwahrscheinlich, dass es solche gibt)
    - **W0LFK>0:**
      - um zu vermeiden, dass versucht wird, FWVB mit der Soll-Leistung 0 zu zeichnen (pFWVBAttribute default is 'W0LFK')

FWVB Attribute (Size, z-Order)

- pFWVBAttribute: columnName (default: 'W0LFK')
  - the column must be able to be converted to a float
  - the conversion is done before FilterFunction
  - **see ApplyFunction and NaNValue for conversion details:**
    - **pFWVBAttributeApplyFunction: Function to be applied to column pFWVBAttribute**
      - default: lambda x: pd.to_numeric(x,errors='coerce')
    - **pFWVBAttributeApplyFunctionNaNValue: Value for NaN-Values produced by pFWVBAttributeA**
      - default: 0
      - .fillna(pFWVBAttributeApplyFunktionNaNValue).astype(float) is called after ApplyFunction

- pFWVBAttributeAsc: z-Order (default: False d.h. “kleine auf große”)
- pFWVBAttributeRefSize: scatter Sy-Area in pts^2 of for RefSizeValue (default: 10**2)
  - corresponding RefSizeValue is Attribute.std() or Attribute.mean() if Attribute.std() is < 1

FWVB (plot only large (small, medium) FWVB . . .)

- quantil_pFWVBAttributeHigh <= (default: 1.)
- quantil_pFWVBAttributeLow >= (default: .0)
- default: all FWVB are plotted
- note that Attribute >0 is a precondition

FWVB Measure (Color)

- pFWVBMeasure (default: ‘FWVB~*~*~*~W’)
  - float() must be possible
- pFWVBMeasureInRefPerc (default: True d.h. Measure wird verarbeitet in Prozent T zu Ref)
  - 0-1
  - if refValue is 0 than refPerc-Result is set to 1
- pFWVBMeasureAlpha/Colormap/Clip
- 3Classes
  - **pFWVBMeasure3Classes (default: False)**
    - **False:**
      - Measure wird nicht in 3 Klassen dargestellt
      - die Belegung von MCategory gemaess FixedLimitsHigh/Low erfolgt dennoch
  - CatTexts (werden verwendet wenn 3Classes Wahr gesetzt ist)
    - für CBLegend (3Classes) als _zusätzliche Beschriftung rechts
    - als Texte für die Spalte MCategory in return pFWVB
    - pMCatTopText
    - pMCatMidText
    - pMCatBotText
  - CatAttribs (werden verwendet wenn 3Classes Wahr gesetzt ist)
    - für die Knotendarstellung
    - pMCatTopAlpha/Color/Clip
    - pMCatMidAlpha/Colormap/Clip
    - pMCatBotAlpha/Color/Clip
- CBFixedLimits
  - pFWVBMeasureCBFixedLimits (default: False d.h. Farbskala nach vorh. min./max. Wert)
    - wird Wahr gesetzt sein, wenn 3Classes Wahr gesetzt ist

        * damit die mittlere Farbskala den Klassengrenzen "gehorcht"
    - pFWVBMeasureCBFixedLimitLow (default: .10)
    - pFWVBMeasureCBFixedLimitHigh (default: .95)

CB

- CBFraction: fraction of original axes to use for colorbar (default: 0.05)
- CBHpad: fraction of original axes between colorbar and new image axes (default: 0.0275)
- CBLabelPad (default: -50)
- CBTicklabelsHPad (default: 0.)
- CBAspect: ratio of long to short dimension (default: 10.)
- CBShrink: fraction by which to shrink the colorbar (default: .3)
- CBAnchorHorizontal: horizontaler Fußpunkt der colorbar in Plot-% (default: 0.)
- CBAnchorVertical: vertikaler Fußpunkt der colorbar in Plot-% (default: 0.2)

CBLegend (3Classes) - Parameterization of the representative Symbols

- CBLe3cTopVPad (default: 1+1*1/4)
- CBLe3cMidVPad (default: .5)
- CBLe3cBotVPad (default: 0-1*1/4)
    - 1 is the height of the Colorbar
    - the VPads (the vertical Sy-Positions) are defined in cax.transAxes Coordinates
    - cax is the Colorbar Axes
- CBLe3cSySize=10**2 (Sy-Area in pts^2)
- CBLe3cSyType='o'

ROHR

- **pROHRFilterFunction: Filterfunction to be applied to PIPEs to determine the PIPEs to be plotted**
    - **default: lambda df: (df.KVR.astype(int).isin([2])) & (df.CONT_ID.astype(int).isin([1001])) & (df.DI>**
        * **KVRisIn: [2]**
            · 1: supply-line
            · 2: return-line
        * **CONT_IDisIn: [1001]**
            · um zu vermeiden, dass Rohre aus Bloecken gezeichnet werden (deren Koordinaten nicht zu den Koordinaten von Rohren aus dem Ansichtsblock passen)
        * **DI>0:**
            · um zu vermeiden, dass versucht wird, Rohre mit dem Innendurchmesser 0 zu zeichnen (pROHRAttribute default is 'DI')

ROHR (PIPE-Line: Size and Color, z-Order)

- **pROHRAttribute: columnName (default: 'DI')**

  - the column must be able to be converted to a float
  - the conversion is done before FilterFunction
  - **see ApplyFunction and NaNValue for conversion details:**
    - **pROHRAttributeApplyFunction: Function to be applied to column pROHRAttribute**
      - default: lambda x: pd.to_numeric(x,errors='coerce')
    - **pROHRAttributeApplyFunctionNaNValue: Value for NaN-Values produced by pROHRAttribut**
      - default: 0
      - .fillna(pROHRAttributeApplyFunktionNaNValue).astype(float) is called after ApplyFunction
- pROHRAttributeAsc: z-Order (default: False d.h. "kleine auf grosse")
- pROHRAttributeLs (default: '-')
- pROHRAttributeRefSize: plot linewidth in pts for RefSizeValue (default: 1.0)
  - corresponding RefSizeValue is Attribute.std() or Attribute.mean() if Attribute.std() is < 1
- pROHRAttributeColorMap (default: plt.cm.binary)
- pROHRAttributeColorMapUsageStart (default: 1./3; Wertebereich: [0,1])
  - Farbskala nach vorh. min./max. Wert
  - die Farbskala wird nur ab UsageStart genutzt
  - d.h. Werte die eine "kleinere" Farbe haetten, bekommen die Farbe von UsageStart

ROHR (plot only large (small, medium) pipes . . .)

- quantil_pROHRAttributeHigh <= (default: 1.)
- quantil_pROHRAttributeLow >= (default: .75)
- default: only the largest 25% are plotted
- note that Attribute >0 is a precondition

ROHR (PIPE-Marker: Size and Color)

- pROHRMeasure columnName (default: 'ROHR~*~*~*~QMAV')
- pROHRMeasureApplyFunction: Function to be applied to column pROHRMeasure (default: lambda x: math.fabs(x))
- pROHRMeasureMarker (default: '.')
- pROHRMeasureRefSize: plot markersize for RefSizeValue in pts (default: 1.0)
  - corresponding RefSizeValue is Measure.std() or Measure.mean() if Measure.std() is < 1
  - if pROHRMeasureRefSize is None: plot markersize will be plot linewidth
- pROHRMeasureColorMap (default: plt.cm.cool)
- pROHRMeasureColorMapUsageStart (default: 0.; Wertebereich: [0,1])
  - Farbskala nach vorh. min./max. Wert
  - die Farbskala wird nur ab UsageStart genutzt

  - d.h. Werte die eine "kleinere" Farbe hätten, bekommen die Farbe von UsageStart

NRCVs - NumeRiCal Values to be displayed

- **pFIGNrcv: List of Sir3sID RegExps to be displayed (i.e. ['KNOT~PKON-Knoten~S*~S+~QM']) default:** the 1st Match is used if a RegExp matches more than 1 Channel

  further Examples for RegExps (and corresponding Texts):

  - WBLZ~WärmeblnzGes~S*~S+~WES (Generation)
  - WBLZ~WärmeblnzGes~S*~S+~WVB (Load)
  - WBLZ~WärmeblnzGes~S*~S+~WVERL (Loss)

  WBLZ~[S]+~S*~S+~S+: Example for a RegExp matching all Channels with OBJTYPE WBLZ

- pFIGNrcvTxt: corresponding (same length required!) List of Texts (i.e. ['Kontrolle DH']) default: None
- **pFIGNrcvFmt (i.e. '{:12s}: {:8.2f} {:6s}')**
  - Text (from pFIGNrcvTxt)
  - Value
  - UNIT (determined from Channel-Data)
- **pFIGNrcvPercFmt (i.e. ' {:6.1f}%')**
  - ValueInRefPercent
  - if refValue==0: 100%
- pFIGNrcvXStart (.5 default)
- pFIGNrcvYStart (.5 default)

Category - User Heat Balances to be displayed

- pFWVBGCategory: List of Heat Balances to be displayed (i.e. ['BLNZ1u5u7']) default: None
- pFWVBGCategoryUnit: Unit of all these Balances (default: '[kW]'])
- pFWVBGCategoryXStart (.1 default)
- pFWVBGCategoryYStart (.9 default)
- **pFWVBGCategoryCatFmt (i.e. '{:12s}: {:6.1f} {:4s}')**
  - Category NAME
  - Category Load
  - pFWVBGCategoryUnit
- **pFWVBGCategoryPercFmt (i.e. ' {:6.1f}%')**
  - Last Ist/Soll
- **pFWVBGCategory3cFmt (i.e. ' {:5d}/{:5d}/{:5d}')**
  - NOfTops
  - NOfMids
  - NOfBots

VICs - VeryImportantCustomers whose Values to be displayed

- **pVICsDf: DataFrame with VeryImportantCustomers (Text & Specification)**

  columns expected:

  - Kundenname (i.e. 'VIC1') - Text
  - Knotenname (i.e. 'V-K007') - Specification by Supply-Node

  i.e.: pd.DataFrame({'Kundenname': ['VIC1'],'Knotenname': ['V-K007']})

- **pVICsPercFmt (i.e. '{:12s}: {:6.1f}%')**
  - Kundenname
  - Load in Percent to Reference
- **pVICsFmt (i.e. '{:12s}: {:6.1f} {:6s}')**
  - Kundenname
  - Load
  - pFWVBGCategoryUnit
- pVICsXStart (.5 default)
- pVICsYStart (.1 default)

Figure:

- pltTitle: title [not suptitle] (default: 'pltNetFigAx')
- figFrameon: figure frame (background): displayed or invisible (default: True)
- figEdgecolor: edge color of the Figure rectangle (default: 'black')
- figFacecolor: face color of the Figure rectangle (default: 'white')

Returns:

pFWVB

- **columns changed (compared to vFWVB):**
  - pFWVBAttribute (wg. z.B. pFWVBAttributeApplyFunction und .astype(float))
- **columns added (compared to vFWVB):**
  - Measure (in % zu Ref wenn pFWVBMeasureInRefPer=True)
  - MeasureRef (Wert von Measure im Referenzzustand)
  - MeasureOrig (Wert von Measure)
  - **MCategory: str (Kategorisierung von Measure mit FixedLimitHigh/Low-Werten):**
    - TopText or
    - MidText or
    - BotText
  - GCategory: list (non-empty only if req. GCategories are a subset of the available Categories and object belongs to a req. Category)
  - VIC (filled with Kundenname from pVICsDf)

- **rows (compared to vFWVB):**
  - pFWVB enthaelt dieselben Objekte wie vFWVB
  - aber: die geplotteten Objekte sind ggf. nur eine Teilmenge (wg. z.B. pFWVBFilterFunction)

exception Rm.**RmError**(*value*)

CHAPTER

FIVE

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

m

Mx, **??**

r

Rm, **??**

x

Xm, **??**